

Toward Practical Lattice-based Unbounded Inner Product Functional Encryption: Construction and Implementation

Suprava Roy^{1*} and Ratna Dutta¹

¹Department of Mathematics, Indian Institute of Technology
Kharagpur, , Kharagpur, 721302, West Bengal, India.

*Corresponding author(s). E-mail(s): supravaroy9@kgpian.iitkgp.ac.in;
Contributing authors: ratna@maths.iitkgp.ac.in;

Abstract

Cloud computing enables data processing, storing and sharing in untrusted environments whose growing adoption necessitates a focus on data security and privacy. *Inner product functional encryption* (IPFE) is a promising cryptographic technique that enables fine-grained access control over sensitive data in untrusted cloud environments. *Post-quantum cryptography* focuses on developing cryptographic protocols resilient to quantum computer attacks, with lattice structures being crucial in designing these protocols.

This paper aims to implement the lattice-based public key unbounded IPFE (uIPFE) scheme proposed by Dutta et al. (2022) [1] which ensures that no specific vector's length bound needs to be fixed during public parameter generation.

Furthermore, we extend the ALS-IPFE scheme of Agrawal et al. (2016) [2] to create a new public key scheme uIPFE #1, achieving adaptive indistinguishability security in the random oracle model under the Learning with Errors assumption, while avoiding trapdoor generation and pre-image sampling algorithms.

This work aims to enhance the practical applicability of uIPFE in cloud computing environments. We implement both the schemes uIPFE and uIPFE #1 in C programming language and execute the code on IBM Power 9 server. Moreover, we analyze the running time and discuss the performance of both schemes based on varying message vector lengths.

Keywords: Inner product functional encryption, unbounded, cloud computing, adaptive, post-quantum security

1 Introduction

The global embrace of smart technology has led to an exceptional surge in electronic data generation. Effectively managing and sharing the vast amount of data generated across various sectors becomes critical with this increase in digital data volume. Cloud servers have appeared as a viable solution for data storage and offer seamless data/message transmission and cost-effectiveness across various sectors, encompassing medicine, digital data storage, social media, large-scale data analytics and e-learning platforms. For instance, the *Internet of Medical Things* (IoMT) enables seamless connectivity between medical devices and cloud-based healthcare services, facilitating remote patient monitoring and predictive analytics. The IoMT refers to a network comprising medical devices, software, sensors and systems that work together and integrate with healthcare IT infrastructures. This innovation is transforming the healthcare sector by enabling real-time data collection, continuous monitoring and advanced data analysis. To conduct such analysis effectively, the IoMT system relies on automation tools or services at the central site that process the data while guarantee security and privacy. However, ensuring data privacy while outsourcing computations to the cloud remains a challenge. *Functional Encryption* (FE) [3] addresses this issue by allowing computations on encrypted medical data, enabling secure AI-driven analysis without exposing sensitive patient information.

FE surpasses the barrier of existing encryption systems that impose an all-or-nothing approach to decryption capabilities. From its inception, FE has offered solutions to many unresolved cryptographic challenges, covering diverse areas such as obfuscation, deniable encryption, distributed protocols and beyond. The fundamental advantage of FE is the provision of fine-grained access control over message space. Specifically, FE for a function class $\mathcal{F}$ employs a master secret key Msk to generate decryption-key Dk_f for any function $f \in \mathcal{F}$ and a master public key Mpk to create ciphertexts Ct_M corresponding to a message M , ensuring that the recipient learns no more about the message M than $f(M)$ after decrypting Ct_M .

The primary security demand of FE is *collusion resistance* which ensures that a collective set of secret keys cannot collectively divulge any details about the message M except what can be derived from each individual secret key. This concept is defined through an *indistinguishability-based* (IND) security experiment in which any polynomial-time adversary $\mathcal{A}$ selects challenge messages M_0 and M_1 and is provided access to the key extraction oracle. The task of $\mathcal{A}$ is to differentiate between two ciphertexts Ct_{M_0} and Ct_{M_1} . For any function f queried to the key extraction oracle, the adversary $\mathcal{A}$ is required to maintain the condition that $f(M_0) = f(M_1)$ in order to be restricted from trivially differentiating the ciphertexts. In selective IND (Sel-IND) setting, $\mathcal{A}$ is required to commit the challenge messages M_0, M_1 , prior to the commencement of the experiment whereas adaptive IND (Adp-IND) is more robust as it gives $\mathcal{A}$ the freedom to choose M_0, M_1 at any point of the security experiment.

Inner product functional encryption (IPFE) is a specific instance of FE that leverages inner product computations for access control. It is suitable for scenarios where fine-grained management of encrypted data is required for vector-based data representations. Abdalla et al. [4] introduced the notion IPFE where a decryption key Dk_y for a key vector y reveals the inner product $\langle x, y \rangle$ after decrypting a ciphertext Ct_x

corresponding to a message vector $\mathbf{x}$. The ciphertext $\text{Ct}_{\mathbf{x}}$ is generated using a master public key Mpk and the decryption key $\text{Dk}_{\mathbf{y}}$ is derived using master secret key Msk . The master public-secret key pair (Mpk, Msk) is generated during **Setup** by a key generation centre. The authors provided several IPFEs in **Sel-IND** model based on the standard assumptions. Later, the **Sel-IND** security was improved by Agrawal et al. [2] to **Adp-IND** security and IPFE constrained was proposed relying on the standard assumptions achieving **Adp-IND** security. In recent years, substantial research has been dedicated to these frameworks through various techniques. In case of IPFEs, this includes improved security guarantees (e.g. [2, 5, 6]), multi-users extension (e.g. [7, 8]) and unbounded size vectors (e.g. [9, 10]).

The majority of IPFEs are classified as ‘*bounded*’ since they can solely perform inner product computations on vectors of fixed lengths predefined during the generation of public parameters. Since the size of public parameters in a bounded IPFE increases proportionally with the magnitude of vectors, leading to an unnecessary efficiency decline. Constructing an *unbounded* IPFE (uIPFE) is gaining attraction in research community since it makes it possible to generate public parameters without having any previous information regarding the magnitude of any vector. This indicates that the size of the master secret key and master public key does not depend on the length of the input vectors. Concurrently, Takashima et al. [9] and Dufour-Sans et al. [10] introduced the unbounded feature within the framework of IPFE. It is worth noting that both constructions attained **IND** security by leveraging pairing-based assumptions. In the literature, two distinct concepts regarding unbounded vectors exist: strict and permissive. The strict notion guarantees that the computation of inner product is viable only when the index sets associated with the message and key vectors are identical. On the other hand, in a permissive scenario, computation becomes feasible exclusively when the index set of the key vector is contained in the index set of the message vector.

To understand how an IPFE works, let us consider an example. In IoMT, hospitals and health monitoring devices continuously collect sensitive patient data, such as heart rate, glucose levels and ECG readings. However, privacy concerns prevent hospitals from directly sharing raw patient data with cloud-based AI services for predictive analysis. Patient health data is represented as a vector $\mathbf{x}$, encrypted employing an IPFE scheme to generate a ciphertext $\text{Ct}_{\mathbf{x}}$ using a master public key Mpk and sent to the cloud. A hospital or AI provider defines a weight vector $\mathbf{y}$ corresponding to a machine learning model. The functional key or decryption key $\text{Dk}_{\mathbf{y}}$ for $\mathbf{y}$ is generated by using the master secret key Msk and provided to the cloud server. The Msk and Mpk are generated by the key generation centre in the **Setup** phase. To obtain AI-based health insights, the cloud server evaluates the inner product computation on encrypted data without learning the sensitive health data $\mathbf{x}$. The unbounded property of uIPFE ensures that the sizes of $\mathbf{x}$ and $\mathbf{y}$ do not need to be fixed beforehand as required in this is more realistic scenario since patient health data $\mathbf{x}$ may not fully known at the beginning of the treatment. In Figure 1, the patient’s health sheet, denoted as $\mathbf{x}_i$, is continuously updated as the treatment progresses. The health components, represented by $\mathbf{y}_i$, are assigned specific weights determined by Clinical and Laboratory Standards Institute (CLSI). As uIPFE allows generation of the ciphertext and secret

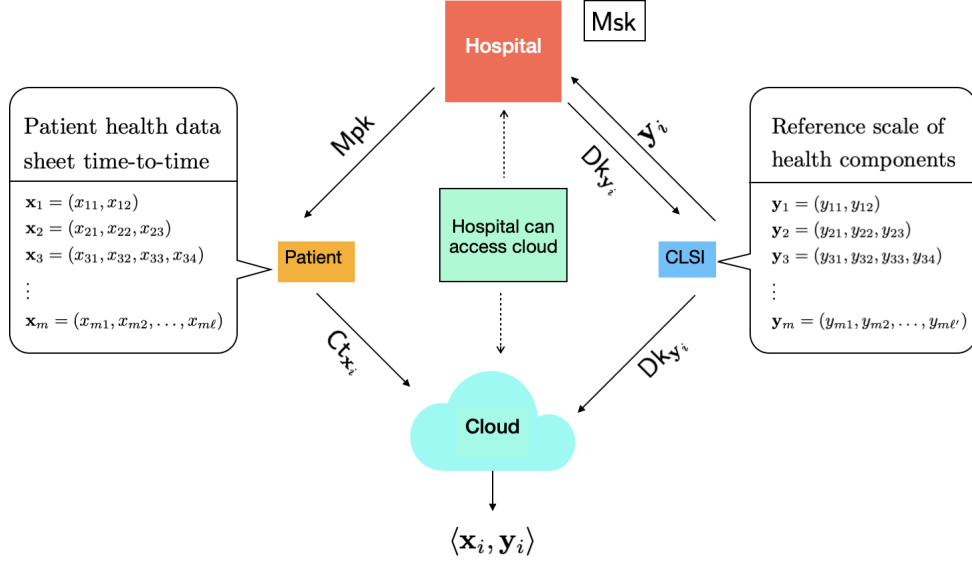


Fig. 1: Application of uIPFE in IoMT

keys for all updated versions are encrypted using the *same* master public-secret key pair (Mpk , Msk), it is an ideal candidate in computing the required statistics while accommodating variations in data size over time. Consequently, unbounded property enhances efficiency and makes IPFE more user-friendly. Without this unbounded feature, encrypting health parameters using IPFE would require frequent updates of (Mpk , Msk) pair by the key generation centre.

All existing works [9, 10], except [1] based on uIPFE relies on discrete logarithm-based assumptions such as symmetric external Diffie-Hellman (SXDH) and decisional bilinear Diffie-Hellman assumption (DBDH), making them vulnerable to quantum attacks. Considering the practical significance of uIPFE and the growing threat of quantum attacks, our objective is to develop efficient post-quantum resistant uIPFE based on learning with errors (LWE) assumptions, an area that had not been explored prior to the work of Dutta et al. [1].

Our Contribution. In this work, we aim to implement the uIPFE of [1] based on the standard LWE assumption. The uIPFE was originally introduced by Dutta et al. in [1] and later was published in [11]. In addition, we have designed and implemented a new public-key unbounded IPFE scheme, referred to as uIPFE #1, that eliminates the need for both trapdoor generation and preimage sampling algorithms. Both the schemes are strict in the sense that they ensure successful decryption when the index set of the message coincides with the secret key vector. From a practical standpoint, the uIPFE

protocol is particularly fruitful in scenarios where the message length is unknown in advance as it allows inner product computation for input vectors of arbitrary size.

Table 1 furnishes a comparative analysis of uIPFE [1] and our scheme uIPFE #1 with other existing uIPFE schemes. The uIPFE [1] and our uIPFE #1 rely on the LWE assumption whereas other constructions depend on number theoretic assumptions like DBDH and SXDH assumptions. We consider implementation of the uIPFE of Dutta et al. [1] as it is the first lattice-based uIPFE scheme, more efficient regarding dimensions of ciphertext and decryption key than the prior works. To be more specific, the approach in uIPFE [1] necessitates ciphertexts of linear size and secret keys of constant size without the use of any bilinear maps and exponentiation operations. Pairing operations are widely recognized to involve complex arithmetic over large extension fields and multiple sub-operations such as elliptic curve multiplications and final exponentiations, making them inherently intensive. In contrast, the uIPFE [1] and our uIPFE #1 are more flexible, efficient and demand significantly lower computations. To date, public-key uIPFE constructions have exclusively relied on bilinear maps and the development of pairing-free schemes grounded in standard cryptographic assumptions remains an open and largely unexplored direction. In the implementation point of view, we observe that there are some implementations on pairing-based IPFE schemes including uIPFE like [12], but there is no existing implementation of a post-quantum secure uIPFE. So we choose the first and only existing lattice-based uIPFE for our implementation as lattice-based constructions are quantum-secure. Also for fast execution, we use parallel execution for some parts of our code and we make it as fast as possible. Considering the substantial computational overhead of pairings, the design of pairing-free uIPFE schemes presents a more pragmatic and efficient option. Such schemes can further serve as promising candidates for building lattice-based IPFE constructions, as eliminating pairing operations leads to considerably lower computational costs compared to their pairing-based counterparts.

Our implementation is in C language which mainly involves the implementation of lattice-based algorithms `TrapdoorGen(·)` and `SamplePreimage(·)` and their integration in the uIPFE. The algorithm `TrapdoorGen(·)` is a trapdoor generation algorithm produces a random matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ along with a strong trapdoor $\mathbf{T} \in \mathbb{Z}^{m \times m}$ consisting of short integer vectors satisfying $\mathbf{A}\mathbf{T} = \mathbf{0} \pmod{q}$ where q is a prime number. The matrix $\mathbf{T}$ is often interpreted as a basis for the lattice $\Lambda_q^\perp(\mathbf{A})$ defined by $\mathbf{A}$ where $\mathbf{A}$ serves as a ‘parity-check’ matrix. The algorithm `SamplePreimage(·)` is the pre-image sampling algorithm that uses the trapdoor $\mathbf{T}$ to efficiently compute a short preimage $\mathbf{Z} \in \mathbb{Z}^{m \times \ell}$ for any given target image $\mathbf{U} \in \mathbb{Z}^{n \times \ell}$ such that $\mathbf{A}\mathbf{Z} = \mathbf{U}$. The uIPFE [1] is adaptively indistinguishable secure in the random oracle model under the hardness problem of LWE. They leverage the security of the IBE of Gentry et al. [13] to handle all secret keys that do not decrypt the challenge ciphertext and employ the proof techniques from [2, 6] to handle the secret keys that successfully decrypt the challenge ciphertext. A high level overview of this uIPFE is described below in subsection 2.4. The setup phase generates system parameter $n, q, m, \ell_{\max}, X, Y, K, \psi, \alpha, \sigma$ and executes the `TrapdoorGen( $n, q, m, \psi$ )` [14] to generate the pair $(\mathbf{A}, \mathbf{T}) \in \mathbb{Z}_q^{n \times m} \times \mathbb{Z}^{m \times m}$ and sets $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ as the master public key and $\mathbf{T} \in \mathbb{Z}^{m \times m}$ as the corresponding

Table 1: A comparative analysis of various existing IPFE schemes

IPFE schemes	$ \text{Msk} $	$ \text{Ct} $	$ \text{Dk} $	Total Operations	Assum.	Security	ROM / SM
[10]	$ \mathbb{G}_1 + \mathbb{Z}_q $	$(I +1) \mathbb{G} $	$ I' \mathbb{Z}_q + \mathbb{G}_2 $	$(2 I + 1) \cdot p + (2 I + I' + 1) \cdot m + (2 I + I' + 2) \cdot e$	DBDH	Sel	ROM
	$ \mathbb{G}_1 + \mathbb{Z}_q $	$(I +1) \mathbb{G} $	$ I' \mathbb{Z}_q + \mathbb{G}_2 $	$(2 I + 1) \cdot p + (2 I + I' + 1) \cdot m + (2 I + I' + 2) \cdot e$	Lin. Ext. DBDH	Sel	ROM
[9]	PRF key	$4 I \mathbb{G}_1 $	$4 I' \mathbb{G}_2 $	$(I') \cdot p + (16 I + 16 I') \cdot m + (4 I + 4 I') \cdot e$	SXDH	Adp-FH	SM
	$28 \cdot \mathbb{Z}_q $	$7 I \mathbb{G}_1 $	$7 I' \mathbb{G}_2 $	$(I') \cdot p + (49 I + 49 I') \cdot m + (7 I + 7 I' + 28) \cdot e$	SXDH	Adp	SM
[1]	$\mathcal{O}(1)$	$(I +m) \mathbb{Z}_q $	$(I' +m) \mathbb{Z}_q $	$(mn + m + n I + m I' + I + I') \cdot m + 0 \cdot p + 0 \cdot e + t + s$	LWE	Adp	ROM
Our work	$\mathcal{O}(1)$	$(I +m) \mathbb{Z}_q $	$(I' +m) \mathbb{Z}_q $	$(mn + m + (n I + m I' + mn)\ell_{\max} + I + I') \cdot m + 0 \cdot p + 0 \cdot e + 0 \cdot t + 0 \cdot s$	LWE	Adp	ROM

6

– Adp, FH, Sel: adaptive, functional-hiding and selective respectively.

– Lin. Ext.: Linearly Extended.

– DBDH, SXDH: decisional bilinear Diffie-Hellman and symmetric external Diffie-Hellman assumptions respectively.

– $|\text{Msk}|, |\text{Ct}|, |\text{Dk}|$: the size of the master secret key, ciphertext and decryption key respectively.

– $|I|, |I'|$: magnitude of the encrypted vector and the decryption key vector respectively.

– $\ell_{\max}$: maximum magnitude of the encrypted vector and the decryption key vector.

– q : a large prime.

– m, n : order of matrix.

– $|\mathbb{G}|, |\mathbb{G}_1|, |\mathbb{G}_2|, |\mathbb{Z}_q|$: size of the groups $\mathbb{G}, \mathbb{G}_1, \mathbb{G}_2, \mathbb{Z}_q$ and size of pseudorandom key respectively.

– p, m, e, t, s : number of pairings, multiplications, exponents, trapdoor generation and preimage sampling operations respectively.

– SM, ROM: standard model and random oracle model respectively.

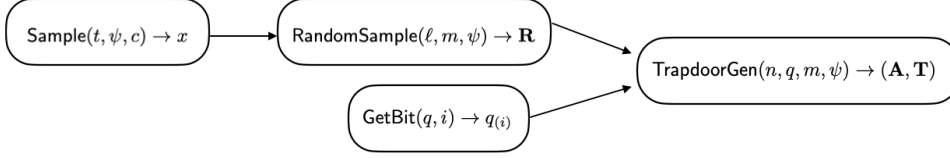


Fig. 2: Basic Overview of algorithm TrapdoorGen

master secret key. The trapdoor $\mathbf{T}$ serves as a short basis for the lattice $\Lambda_q^\perp(\mathbf{A})$ satisfying $\mathbf{A}\mathbf{T} = \mathbf{0} \bmod q$ where q is a prime and $m = n\lceil\log q\rceil$. A full domain hash function $\mathcal{H} : \mathbb{N} \rightarrow \mathbb{Z}_q^n$ is used which is modeled as a random oracle. For encryption, the indices of the message vector $\mathbf{x} = (x_i)_{i \in I_{\mathbf{x}}} \in \mathbb{Z}_q^{|I_{\mathbf{x}}|}$ where $\mathbf{u}_i = \mathcal{H}(I_{\mathbf{x}}[i]) \in \mathbb{Z}_q^n$ and $I_{\mathbf{x}}[i]$ is the i -th element of the index set $I_{\mathbf{x}} \subset \mathbb{N}$ of $\mathbf{x}$, are used to construct a matrix $\mathbf{U} = [\mathbf{u}_1 \| \mathbf{u}_2 \| \dots \| \mathbf{u}_{|I_{\mathbf{x}}|}] \in \mathbb{Z}_q^{n \times |I_{\mathbf{x}}|}$. A column vector $\mathbf{c}_1 = \mathbf{A}^\top \mathbf{s} + \mathbf{e}_1 \in \mathbb{Z}_q^n$ is set as the first ciphertext component where $\mathbf{s}$ is uniformly random column vectors in $\mathbb{Z}_q^n$ and $\mathbf{e}_1$ is selected at random from discrete Gaussian distribution $\mathfrak{D}_{\mathbb{Z}^m, \psi}$ with ψ is Gaussian parameter. a column vector $\mathbf{c}_2 = \mathbf{U}^\top \mathbf{s} + \lfloor \frac{q}{K} \rfloor \mathbf{x} + \mathbf{e}_2 \in \mathbb{Z}_q^{|I_{\mathbf{x}}|}$ where K is set as polynomially constrained in the security parameter λ , is used to generate the second ciphertext component. The decryption key related to secret key column vector $\mathbf{y} = (y_i)_{i \in I_{\mathbf{y}}} \in \mathbb{Z}_q^{|I_{\mathbf{y}}|}$ is set as $\text{Dk}_{\mathbf{y}} = (\mathbf{y}^\top, \mathbf{y}^\top \mathbf{Z}^\top) \in \mathbb{Z}_q^{1 \times |I_{\mathbf{y}}|} \times \mathbb{Z}_q^{1 \times m}$ where $\mathbf{Z} \in \mathbb{Z}^{m \times |I_{\mathbf{y}}|}$ is produced by using the preimage sampling algorithm $\text{SamplePreimage}(n, q, \mathbf{A}, \mathbf{T}, \sigma, \mathbf{U}', \ell')$ [15] with $\mathbf{U}' = [\mathbf{u}'_1 \| \mathbf{u}'_2 \| \dots \| \mathbf{u}'_{|I_{\mathbf{y}}|}] \in \mathbb{Z}^{n \times |I_{\mathbf{y}}|}$, $\mathbf{u}'_i = \mathcal{H}(I_{\mathbf{y}}[i]) \in \mathbb{Z}_q^n$, $I_{\mathbf{y}}[i]$ is the i -th element of the index set $I_{\mathbf{y}} \subset \mathbb{N}$ of $\mathbf{y}$ using the master secret key $\mathbf{T}$ with σ as a Gaussian parameter and $\mathbf{Z} \in \mathbb{Z}^{m \times |I_{\mathbf{y}}|}$ satisfying $\mathbf{U}' = \mathbf{A}\mathbf{Z} \in \mathbb{Z}^{n \times |I_{\mathbf{y}}|}$. The decryptor can use the decryption key $\text{Dk}_{\mathbf{y}} = (\mathbf{y}^\top, \mathbf{y}^\top \mathbf{Z}^\top) \in \mathbb{Z}_q^{1 \times |I_{\mathbf{y}}|} \times \mathbb{Z}_q^{1 \times m}$ to recover the inner product $\langle \mathbf{x}, \mathbf{y} \rangle$ from the ciphertext $\text{Ct}_{\mathbf{x}} = (\mathbf{c}_1, \mathbf{c}_2)$ if $I_{\mathbf{x}} = I_{\mathbf{y}}$ by computing

$$\mu' = \mathbf{y}^\top \mathbf{c}_2 - \mathbf{y}^\top \mathbf{Z}^\top \mathbf{c}_1 \approx \langle \mathbf{x}, \mathbf{y} \rangle + \text{noise}.$$

Our implementation efficiently sets the parameters to minimize the noise function. We carefully select the parameters that satisfy the necessary conditions for managing the noise function effectively. We emphasize a program for parameter choice described in subsection 3.2. If the error quantity $\text{noise} \leq \sigma \psi m \ell_{\max} Y + Y \ell_{\max} \psi \leq \frac{q}{2K}$, the decryption will produce the correct result i.e. $q \geq 2K(\sigma \psi m \ell_{\max} Y + Y \ell_{\max} \psi)$.

We now turn the attention to outline the implementation aspects used for our experimental validation of the uIPFE [1] and the challenges we found. Note that the performance of the protocol is mainly affected by the procedures TrapdoorGen and SamplePreimage. We implement the algorithms TrapdoorGen and SamplePreimage in C language and execute in the IBM power 9 server. Subsequently, we integrate these into our implementation of the uIPFE [1]. For our implementation, the trapdoor generation algorithm TrapdoorGen is adopted from the trapdoor generation algorithm of Micciancio et al. [14] and the Gaussian preimage sampling SamplePreimage is adopted from the works of Bert et al. [15] and Gur et al. [16]. We provide pseudocodes of

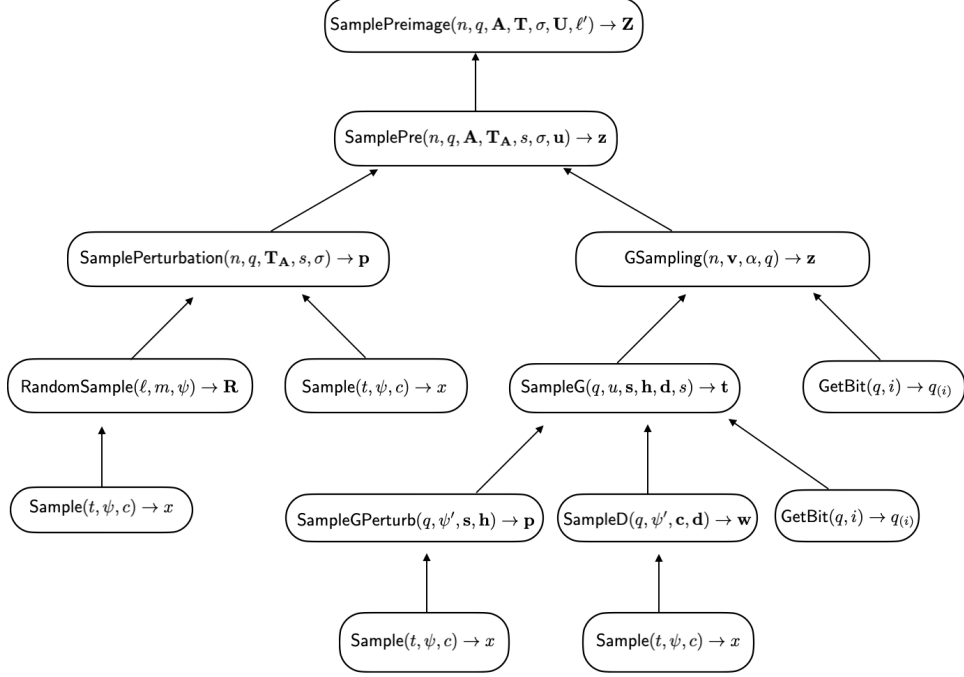


Fig. 3: Basic Overview of algorithm `SamplePreimage`

these algorithms and discuss the conditions needed to choose their parameters. Figures 2 and 3 depict a rough outline of the algorithms `TrapdoorGen` and `SamplePreimage` respectively. The algorithm `TrapdoorGen`(n, q, m, ψ) $\rightarrow (\mathbf{A}, \mathbf{T})$ uses procedures `RandomSample`(ℓ, m, ψ) $\rightarrow \mathbf{R}$ and `GetBit`(q, i) $\rightarrow q_{(i)}$ as subroutines to generate a random matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and a trapdoor basis $\mathbf{T} \in \mathbb{Z}^{m \times m}$ for lattice $\Lambda_q^\perp(\mathbf{A})$. The procedure `RandomSample`(ℓ, m, ψ) $\rightarrow \mathbf{R}$ generates random matrix $\mathbf{R}$ of order $\ell \times m$ sampled from discrete Gaussian distribution $\mathcal{D}_{\mathbb{Z}^{\ell \times m}, \psi}$ using a Gaussian sampler `Sample`(t, ψ, c) $\rightarrow x$ as subroutine. The procedure `Sample`(t, ψ, c) $\rightarrow x$, adapted from the work of Ducas et al. [17] is sampler of discrete Gaussian distributions over $\mathbb{Z}$. The procedure `GetBit`(q, i) $\rightarrow q_{(i)}$ returns i -th bit of a given integer q . The algorithm `SamplePreimage`($n, q, \mathbf{A}, \mathbf{T}, \sigma, \mathbf{U}, \ell'$) $\rightarrow \mathbf{Z}$ generates a matrix $\mathbf{Z} \in \mathbb{Z}^{m \times \ell'}$ by running a procedure `SamplePre`($n, q, \mathbf{A}, \mathbf{T}_\mathbf{A}, s, \sigma, \mathbf{u}$) $\rightarrow \mathbf{z}$ as subroutine to generate each column vector $\mathbf{z} \in \mathbb{Z}^m$ of the matrix $\mathbf{Z} \in \mathbb{Z}^{m \times \ell'}$ which can be run in parallel, given a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, its trapdoor basis $\mathbf{T} \in \mathbb{Z}^{m \times m}$ and a matrix $\mathbf{U} \in \mathbb{Z}^{n \times \ell'}$ such that $\mathbf{AZ} = \mathbf{U} \bmod q$. The procedure `SamplePre`($n, q, \mathbf{A}, \mathbf{T}_\mathbf{A}, s, \sigma, \mathbf{u}$) $\rightarrow \mathbf{z}$ calls `SamplePerturbation`($n, q, \mathbf{T}_\mathbf{A}, s, \sigma$) $\rightarrow \mathbf{p}$ and `GSampling`($n, \mathbf{v}, \alpha, q$) $\rightarrow \mathbf{z}$ as subroutines to generate each column vector $\mathbf{z} \in \mathbb{Z}^m$ of the matrix $\mathbf{Z} \in \mathbb{Z}^{m \times \ell'}$, given matrices $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, $\mathbf{T}_\mathbf{A} \in \mathbb{Z}^{\overline{m} \times \omega}$ and column vector $\mathbf{u} \in \mathbb{Z}^n$ of matrix $\mathbf{U} \in \mathbb{Z}^{n \times \ell'}$ such that $\mathbf{Az} = \mathbf{u} \bmod q$. The Perturbation sampling algorithm `SamplePerturbation`($n, q, \mathbf{T}_\mathbf{A}, s, \sigma$) $\rightarrow \mathbf{p}$, aimed at generating vectors that adhere to a discrete Gaussian distribution on

$\mathbb{Z}^m$ with a covariance dependent on the trapdoor $\mathbf{T} \in \mathbb{Z}^{m \times m}$, was developed by Genise and Micciancio [18] and modified by Gur et al. [16]. On the other hand, $\text{GSampling}(n, \mathbf{v}, \alpha, q) \rightarrow \mathbf{z}'$ computes a vector $\mathbf{z}' \in \mathbb{Z}^\omega$ following a discrete Gaussian distribution of parameter s over a coset $\Lambda_q^{\mathbf{v}}(\mathbf{G})$ of the G-lattice $\Lambda_q^\perp(\mathbf{G})$ given an n -dimensional target vector $\mathbf{v} \in \mathbb{Z}^m$ by using the procedure $\text{SampleG}(q, u, \mathbf{s}, \mathbf{h}, \mathbf{d}, s) \rightarrow \mathbf{t}$. The procedure $\text{SampleG}(q, u, \mathbf{u}, \mathbf{h}, \mathbf{d}, s) \rightarrow \mathbf{t}$ is designed to work with a single integer component $v_i \in \mathbb{Z}$ of the m -dimensional target vector $\mathbf{v} \in \mathbb{Z}^m$ instead of $\mathbf{v} \in \mathbb{Z}^m$ because each of the m integers can be sampled parallelly by running SampleG inside the function $\text{GSampling}(n, \mathbf{v}, \alpha, q) \rightarrow \mathbf{z}'$ to generate the vector $\mathbf{z}' \in \mathbb{Z}^\omega$. More specifically, this SampleG uses a perturbation sampling procedure $\text{SampleGPerturb}(q, \psi', \mathbf{s}, \mathbf{h}) \rightarrow \mathbf{p}$ and a discrete Gaussian sampling procedure $\text{SampleD}(q, \psi', \mathbf{c}, \mathbf{d}) \rightarrow \mathbf{w}$. The procedures $\text{SampleGPerturb}(q, \psi', \mathbf{s}, \mathbf{h}) \rightarrow \mathbf{p}$ generates a perturbation vector $\mathbf{p} \in \mathbb{Z}^l$ and $\text{SampleD}(q, \psi', \mathbf{c}, \mathbf{d}) \rightarrow \mathbf{w}$ returns a sample $\mathbf{w} \in \mathbb{Z}^l$. Both the procedures SampleD and SampleGPerturb rely on an integer sampling subroutine $\text{Sample}(t, \psi, c) \rightarrow x$ which yields a sample statistically approximating a one-dimensional discrete Gaussian with a tail-cut t , parameter ψ and center c . Our implementation supports multi-threaded execution for parallel computation of the procedure SamplePre inside the algorithm SamplePreimage and the procedure SampleG inside GSampling . We experimentally evaluate multi-threaded performance of the uIPFE [1] on IBM Power9 server and provide the performance of the algorithms in section 5.

Our second contribution is construction of a strict unbounded inner product functional encryption scheme uIPFE #1. We propose a public-key uIPFE scheme based on LWE in the random oracle model with adaptively indistinguishability (Adp-IND) security. Our scheme achieves Adp-IND security in ROM without using any trapdoor generation algorithm and pre-image sampling algorithm. Thus our construction not only reduces the time complexity but also makes it simple and easy to implement. In the setup phase, we choose a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ uniformly at random and sample a matrix $\mathbf{Z} \in \mathcal{D}_{\mathbb{Z}^m \times \ell_{\max}, \sigma}$ from discrete Gaussian distribution with parameter σ and compute $\mathbf{W} = \mathbf{AZ} \in \mathbb{Z}_q^{n \times \ell_{\max}}$ where $\ell_{\max}$ is the maximum length of the message vectors and secret key vectors. Then we set $\mathbf{A}, \mathbf{W}$ as master public key and $\mathbf{Z}$ as master secret key. We employ a full domain hash function, represented as a random oracle, $\mathcal{H} : \mathbb{N} \rightarrow \{-1, 1\}^{\ell_{\max}}$. The indices of the message vector $\mathbf{x} = (x_i)_{i \in I_{\mathbf{x}}} \in \mathbb{Z}_q^{|I_{\mathbf{x}}|}$ are used to construct a matrix $\mathbf{U} = [\mathbf{u}_1 \| \mathbf{u}_2 \| \dots \| \mathbf{u}_{|I_{\mathbf{x}}|}] \in \mathbb{Z}_q^{\ell_{\max} \times |I_{\mathbf{x}}|}$ for encryption where $\mathbf{u}_i = \mathcal{H}(I_{\mathbf{x}}[i]) \in \mathbb{Z}_q^n$ and $I_{\mathbf{x}}[i]$ is the i -th element of the index set $I_{\mathbf{x}} \subset \mathbb{N}$ of $\mathbf{x}$. A column vector $\mathbf{c}_1 = \mathbf{A}^\top \mathbf{s} + \mathbf{e}_1 \in \mathbb{Z}_q^m$ is set as the first component of the ciphertext where $\mathbf{s}$ is uniformly random column vector in $\mathbb{Z}_q^n$ and $\mathbf{e}_1$ is selected at random from discrete Gaussian distribution $\mathcal{D}_{\mathbb{Z}^m, \psi}$ with ψ is Gaussian parameter. A column vector $\mathbf{c}_2 = \mathbf{U}^\top \mathbf{W}^\top \mathbf{s} + \lfloor \frac{q}{K} \rfloor \mathbf{x} + \mathbf{e}_2 \in \mathbb{Z}_q^{|I_{\mathbf{x}}|}$ is set as the second component of the ciphertext where K is specified as polynomially constrained in the security parameter λ . The decryption key related to secret key column vector $\mathbf{y} = (y_i)_{i \in I_{\mathbf{y}}} \in \mathbb{Z}_q^{|I_{\mathbf{y}}|}$ is set as $\text{Dk}_{\mathbf{y}} = (\mathbf{y}^\top, \mathbf{y}^\top \mathbf{U}'^\top \mathbf{Z}^\top) \in \mathbb{Z}_q^{1 \times |I_{\mathbf{y}}|} \times \mathbb{Z}_q^{1 \times m}$ where $\mathbf{Z} \in \mathcal{D}_{\mathbb{Z}^m \times |I_{\mathbf{y}}|, \sigma}$ with $\mathbf{U}' = [\mathbf{u}'_1 \| \mathbf{u}'_2 \| \dots \| \mathbf{u}'_{|I_{\mathbf{y}}|}] \in \mathbb{Z}_q^{\ell_{\max} \times |I_{\mathbf{y}}|}$, $\mathbf{u}'_i = \mathcal{H}(I_{\mathbf{y}}[i]) \in \mathbb{Z}_q^{\ell_{\max}}$, $I_{\mathbf{y}}[i]$ is the i -th element of the index set $I_{\mathbf{y}} \subset \mathbb{N}$ of $\mathbf{y}$. The decryptor can recover the inner product $\langle \mathbf{x}, \mathbf{y} \rangle$ from the ciphertext $\text{Ct}_{\mathbf{x}} = (\mathbf{c}_1, \mathbf{c}_2)$ using the decryption key

$\text{Dk}_{\mathbf{y}} = (\mathbf{y}^\top, \mathbf{y}^\top \mathbf{U}'^\top \mathbf{Z}^\top) \in \mathbb{Z}_q^{1 \times |I_{\mathbf{y}}|} \times \mathbb{Z}_q^{1 \times m}$ if $I_{\mathbf{x}} = I_{\mathbf{y}}$ by computing

$$\mu' = \mathbf{y}^\top \mathbf{c}_2 - \mathbf{y}^\top \mathbf{U}'^\top \mathbf{Z}^\top \mathbf{c}_1 \approx \langle \mathbf{x}, \mathbf{y} \rangle + \text{noise}.$$

In order to reduce the noise, we effectively set the parameters. The decryption will yield accurate result if the error magnitude $\text{noise} \leq \sigma \psi m Y \ell_{\max}^2 + Y \ell_{\max} \psi \leq \frac{q}{2K}$ i.e. $q \geq 2K(\sigma \psi m Y \ell_{\max}^2 + Y \ell_{\max} \psi)$. For further information, see Section 4 (construction) and Theorem 9 (security).

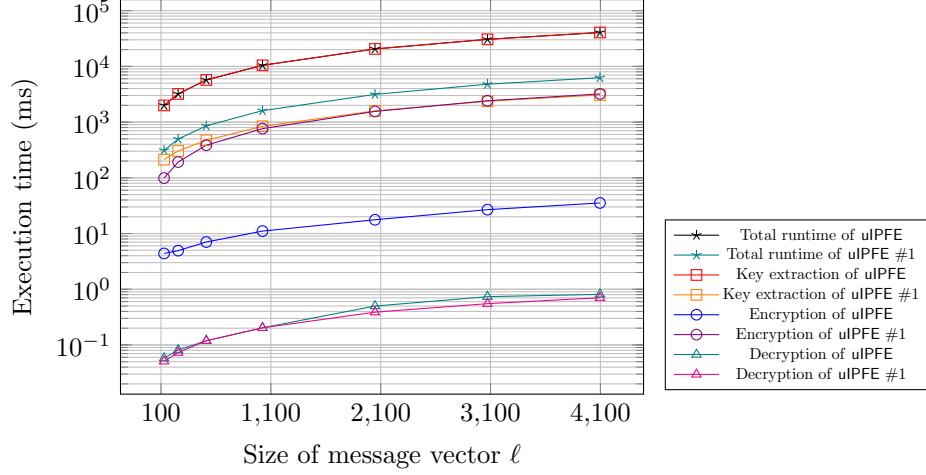


Fig. 4: Execution time comparison of `uIPFE.KeyExtract`, `uIPFE.Encrypt`, `uIPFE.Decrypt` of `uIPFE` [1] and `uIPFE #1` for $\ell = 128, 256, 512, 1024, 2048, 3072, 4096$ with the choice of parameter as $n = 5$, $q \approx 2^{49}$ (`uIPFE`), $q \approx 2^{60}$ (`uIPFE #1`), $m = 490$ (`uIPFE`), $m = 600$ (`uIPFE #1`), $\ell_{\max} = 4096$, $X = Y = 4$, $\psi = 2.236$, $\alpha = 4.495 \times 10^{-14}$ (`uIPFE`), $\alpha = 1.069 \times 10^{-17}$ (`uIPFE #1`), $\sigma = 83.29$

Figure 4 illustrates the comparison of run times of `uIPFE.KeyExtract`, `uIPFE.Encrypt` and `uIPFE.Decrypt` of `uIPFE` and `uIPFE #1` with different values of ℓ based on the particular choices of parameter. We can clearly observe that the total runtime of our `uIPFE #1` is quite less than that of `uIPFE` [1]. In particular, our key extraction and decryption algorithms achieve notably lower execution times, whereas the encryption algorithm in [1] exhibits marginally better performance than ours.

Organization. The subsequent portions of the paper are organized as follows. Section 2 presents a comprehensive introduction to the essential principles that are relevant to our implementation and construction. The algorithms those are needed to implement the scheme `uIPFE` [1] are discussed in subsections 2.4 and 3.1. Section 4 provides the construction and security analysis of our proposed scheme `uIPFE #1`. Section 5 is dedicated for analyzing the performance of the scheme `uIPFE` [1] and `uIPFE #1`. Section 6 serves as the concluding section of the paper.

2 Preliminaries

Notations.

We denote matrices and vectors by upper-case bold letters and lower-case bold letters respectively. We use notation λ to represent the security parameter, while $\text{poly}(\lambda)$ represents a polynomial in λ . The set of integers and set of natural numbers are denoted by $\mathbb{Z}$ and $\mathbb{N}$ respectively. The field $\mathbb{Z}/q\mathbb{Z}$ is denoted by $\mathbb{Z}_q$ where q is a prime. For $m \in \mathbb{N}$, $[m]$ is used to represent the set $\{1, 2, \dots, m\}$. For any set I , $I[i]$ denotes the i -th element of the set. For any real number r , $\text{frac}(r)$ denotes the fractional part of the real number. The notation $s \stackrel{\$}{\leftarrow} S$ indicates that s is uniformly sampled from the set S and ' $\mathbf{A} \parallel \mathbf{B}$ ' indicates the concatenation of $\mathbf{A}$ and $\mathbf{B}$.

Each matrix $\mathbf{M}$ of order $m \times n$ can be represented as $\mathbf{M} = [\mathbf{m}_1 \parallel \mathbf{m}_2 \parallel \dots \parallel \mathbf{m}_n]$ where $\mathbf{m}_i$ denotes the i -th column of the matrix $\mathbf{M}$ and $m_{i,j}$ denotes the element of $\mathbf{M}$ at the i -th row and j -th column. The Gram-Schmidt orthogonalization of the columns of a matrix $\mathbf{T}$ is denoted as $\tilde{\mathbf{T}}$. Transpose of a matrix $\mathbf{M}$ is denoted by $\mathbf{M}^\top$ and the identity matrix of order m is denoted by $\mathbf{I}_m$. Tensor product of two matrices $\mathbf{M}$ of order $m \times n$ and $\mathbf{N}$ of order $r \times s$ is defined by $\mathbf{M} \otimes \mathbf{N} = (m_{i,j} \mathbf{N})_{i \in [m], j \in [n]}$ where $\mathbf{M} = (m_{i,j})_{i \in [m], j \in [n]}$. The ℓ_2 -norm of a vector $\mathbf{u} = (u_1, u_2, \dots, u_n)$ is defined as $\|\mathbf{u}\| = \sqrt{u_1^2 + u_2^2 + \dots + u_n^2}$. The max norm of a matrix $\mathbf{M}$ of order $m \times n$ is defined as $\|\mathbf{M}\| = \max_{1 \leq i \leq n} \|\mathbf{m}_i\|$ where $\|\mathbf{m}_i\|$ denotes the ℓ_2 -norm of the i -th column of $\mathbf{M}$. For a matrix $\mathbf{W} = [\mathbf{w}_1 \parallel \mathbf{w}_2 \parallel \dots \parallel \mathbf{w}_m]$ of order $n \times m$, the matrix 2-norm is defined as $\|\mathbf{W}\|_2 = \sup_{\|\mathbf{x}\|=1} \|\mathbf{W}\mathbf{x}\|$ with the column vector $\mathbf{x} \in \mathbb{Z}_q^m$. We indicate a vector $\mathbf{u} = (u_1, u_2, \dots, u_n) \in \mathbb{Z}^n$ as $\mathbf{u} = (u_i)_{i \in I_{\mathbf{u}}}$ where $u_i \neq 0, \forall i \in I_{\mathbf{u}}, I_{\mathbf{u}} \subset \mathbb{N}$ being the index set of $\mathbf{u}$. The inner product of any two vectors $\mathbf{u} = (u_i)_{i \in I_{\mathbf{u}}}$ and $\mathbf{v} = (v_j)_{j \in I_{\mathbf{v}}}$ is defined as

$$\langle \mathbf{u}, \mathbf{v} \rangle = \sum_{i \in I_{\mathbf{u}} \cap I_{\mathbf{v}}} u_i v_i$$

Note that $u_i = 0$ for $i \notin I_{\mathbf{u}}$ and $v_i = 0$ for $i \notin I_{\mathbf{v}}$. The notation $\mathbf{A} \approx_s \mathbf{B}$ denotes $\mathbf{A}$ and $\mathbf{B}$ are statistically close.

Definition 1 (Negligible Function) A function $\text{negl} : \mathbb{N} \rightarrow \mathbb{R}$ is said to be negligible if for every $c \in \mathbb{N}$, there exists an integer $N \in \mathbb{N}$ such that $\text{negl}(n) \leq n^{-c} \forall n \geq N$.

Definition 2 (PPT Algorithm) An algorithm $\mathcal{A}$ is referred to as a *probabilistic polynomial time* (PPT) algorithm if it functions as a probabilistic model executed by a Turing machine within $\text{poly}(\lambda)$ time.

Definition 3 (Statistically Close) Assume that two random variables, A and B , have values in a finite set C . The *statistical distance* is defined as

$$\Delta(A, B) = \frac{1}{2} \sum_{c \in C} |\Pr[A = c] - \Pr[B = c]|$$

If $\Delta(A(\lambda), B(\lambda))$ is negligible function of λ , then two emsembles of random variables $A(\lambda)$ and $B(\lambda)$ are considered to be *statistically close*.

2.1 Strict Unbounded Inner Product Functional Encryption

A strict unbounded inner product functional encryption (uIPFE) scheme, defined for a message space $\mathcal{X}$ and a key space $\mathcal{Y}$ with strict relation $\mathcal{R}_s : \mathcal{X} \times \mathcal{Y} \rightarrow \{0, 1\}$ defined by $\mathcal{R}_s = \{(\mathbf{x}, \mathbf{y}) \in \mathcal{X} \times \mathcal{Y} \mid I_{\mathbf{x}} = I_{\mathbf{y}}\}$, comprises four PPT algorithms uIPFE = (Setup, KeyExtract, Encrypt, Decrypt) that fulfill the following requirements:

- Setup(1^λ) $\rightarrow$ (Mpk, Msk): A trusted authority produces the master public key Mpk and the master secret key Msk on input a security parameter λ . The Mpk is made publicly available whereas the Msk is kept hidden to the authority itself.
- KeyExtract(Mpk, Msk, $\mathbf{y}$) $\rightarrow$ Dk $_{\mathbf{y}}$: The algorithm is executed by the trusted authority using Mpk, Msk and a secret key vector $\mathbf{y} = (y_i)_{i \in I_{\mathbf{y}}} \in \mathcal{Y}$ as input to produce a decryption key Dk $_{\mathbf{y}}$ for $\mathbf{y}$ where $I_{\mathbf{y}} \subseteq \mathbb{N}$ is the finite index set of $\mathbf{y}$.
- Encrypt(Mpk, $\mathbf{x}$) $\rightarrow$ Ct $_{\mathbf{x}}$: The encryptor executes this algorithm with Mpk as input along with a message vector $\mathbf{x} = (x_i)_{i \in I_{\mathbf{x}}} \in \mathcal{X}$, producing a ciphertext Ct $_{\mathbf{x}}$ for $\mathbf{x}$ where $I \subseteq \mathbb{N}$ represents the finite index set of $\mathbf{x}$.
- Decrypt(Mpk, Dk $_{\mathbf{y}}$, Ct $_{\mathbf{x}}$) $\rightarrow$ $\mu = \langle \mathbf{x}, \mathbf{y} \rangle$ or $\perp$: The decryptor uses Dk $_{\mathbf{y}}$ to decrypt Ct $_{\mathbf{x}}$ using Mpk, producing either the inner product $\mu = \langle \mathbf{x}, \mathbf{y} \rangle$ if $\mathcal{R}_s(\mathbf{x}, \mathbf{y}) = 1$ (i.e. $I_{\mathbf{x}} = I_{\mathbf{y}}$) or a specific symbol $\perp$ to indicate failure.

Correctness: An unbounded inner product functional encryption scheme uIPFE = (Setup, KeyExtract, Encrypt, Decrypt) is said to be *correct* if, for all $\lambda \in \mathbb{N}$ and for all $\mathbf{x} = (x_i)_{i \in I_{\mathbf{x}}}, \mathbf{y} = (y_i)_{i \in I_{\mathbf{y}}}$ with $\mathcal{R}_s(\mathbf{x}, \mathbf{y}) = 1$, i.e. $I_{\mathbf{x}} = I_{\mathbf{y}} = I$ (say), the following holds:

$$\Pr \left[\mu = \langle \mathbf{x}, \mathbf{y} \rangle = \sum_{i \in I} x_i y_i \mid \begin{array}{l} (\text{Mpk}, \text{Msk}) \leftarrow \text{uIPFE.Setup}(1^\lambda); \\ \text{Dk}_{\mathbf{y}} \leftarrow \text{uIPFE.KeyExtract}(\text{Mpk}, \text{Msk}, \mathbf{y}); \\ \text{Ct}_{\mathbf{x}} \leftarrow \text{uIPFE.Encrypt}(\text{Mpk}, \mathbf{x}); \\ \mu \leftarrow \text{uIPFE.Decrypt}(\text{Mpk}, \text{Dk}_{\mathbf{y}}, \text{Ct}_{\mathbf{x}}) \end{array} \right] = 1$$

Definition 4 (Adaptive indistinguishable (Adp-IND)) An unbounded inner product functional encryption scheme uIPFE = (Setup, KeyExtract, Encrypt, Decrypt) is said to be *adaptive indistinguishable* (Adp-IND) secure if, for every PPT adversary $\mathcal{A}$ and for any $\lambda \in \mathbb{N}$, there exists a negligible function $\text{negl}(\cdot)$ such that

$$\text{Adv}_{\mathcal{A}, \text{Adp-IND}}^{\text{uIPFE}}(\lambda) = |\Pr[\text{Expt}_{\mathcal{A}, \text{Adp-IND}}^{\text{uIPFE}}(\lambda) = 1] - \frac{1}{2}| \leq \text{negl}(\lambda)$$

where the experiment $\text{Expt}_{\mathcal{A}, \text{Adp-IND}}^{\text{uIPFE}}(\lambda)$ between a challenger $\mathcal{C}$ and the adversary $\mathcal{A}$ is given below.

1. **Setup:** The challenger $\mathcal{C}$ generates the master public-secret key pair $(\text{Mpk}, \text{Msk}) \leftarrow \text{uIPFE.Setup}(1^\lambda)$ and transmits Mpk to the adversary $\mathcal{A}$.
2. **Phase I:** The adversary can make polynomial number of secret key query to the oracle $\mathcal{O}_{\text{KE}}(\text{Mpk}, \text{Msk}, \cdot)$. This oracle on query a secret key vector $\mathbf{y} \in \mathcal{Y}$ by $\mathcal{A}$, returns a secret key Dk $_{\mathbf{y}}$ to $\mathcal{A}$. It takes input a key vector $\mathbf{y} = (y_i)_{i \in I_{\mathbf{y}}}$ satisfying $\langle \mathbf{x}^{(0)}, \mathbf{y} \rangle = \langle \mathbf{x}^{(1)}, \mathbf{y} \rangle$ whenever $I_{\mathbf{y}} = I^*$ and outputs the secret decryption key $\text{Dk}_{\mathbf{y}} \leftarrow \text{uIPFE.KeyExtract}(\text{Mpk}, \text{Msk}, \mathbf{y})$. The adversary $\mathcal{A}$ sends a pair of messages $(\mathbf{x}^{(0)}, \mathbf{x}^{(1)}) \leftarrow \mathcal{A}^{\mathcal{O}_{\text{KE}}(\text{Mpk}, \text{Msk}, \cdot)}(\text{Mpk})$

- to $\mathcal{C}$ where $\mathbf{x}^{(0)} = (x_i^{(0)})_{i \in I_{\mathbf{x}^{(0)}}} \in \mathcal{X}$, $\mathbf{x}^{(1)} = (x_i^{(1)})_{i \in I_{\mathbf{x}^{(1)}}} \in \mathcal{X}$ with $I_{\mathbf{x}^{(0)}} = I_{\mathbf{x}^{(1)}} = I^*$ (say).
3. **Challenge:** The challenger $\mathcal{C}$ picks uniformly at random $\beta \xleftarrow{\$} \{0, 1\}$, computes the challenge ciphertext $\text{Ct}_{\mathbf{x}^{(\beta)}} \leftarrow \text{uIPFE.Encrypt}(\text{Mpk}, \mathbf{x}^{(\beta)})$ and sends $\text{Ct}_{\mathbf{x}^{(\beta)}}$ to $\mathcal{A}$.
 4. **Phase II:** The adversary $\mathcal{A}$ can make polynomial number of query to $\mathcal{O}_{\text{KE}}(\text{Mpk}, \text{Msk}, \cdot)$ as in **Phase I**.
 5. **Guess:** The adversary outputs a guess bit $\beta' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{KE}}(\text{Mpk}, \text{Msk}, \cdot)}(\text{Mpk}, \text{Ct}_{\mathbf{x}^{(\beta)}})$ after making a polynomial number of secret key query to the oracle $\mathcal{O}_{\text{KE}}(\text{Mpk}, \text{Msk}, \cdot)$ and sends β' to $\mathcal{C}$.
 6. The challenger $\mathcal{C}$ output 1 if $\beta = \beta'$, else 0. The experiment outputs 1 if $\mathcal{C}$ outputs 1, else 0.

2.2 Background Related to Lattices

Lattices[19, 20] Let $\mathbf{B} = [\mathbf{b}_1 \| \mathbf{b}_2 \| \dots \| \mathbf{b}_m] \in \mathbb{R}^{m \times m}$ be a matrix formed by m linearly independent column vectors $\mathbf{b}_1, \mathbf{b}_2, \dots, \mathbf{b}_m \in \mathbb{R}^m$. The lattice $\Lambda \subseteq \mathbb{R}^m$ generated by $\mathbf{B}$ which is of full rank m , is defined as:

$$\Lambda = \mathcal{L}(\mathbf{B}) = \{\mathbf{y} \in \mathbb{R}^m \mid \exists \mathbf{u} = (u_1, u_2, \dots, u_m) \in \mathbb{Z}^m, \mathbf{y} = \mathbf{B}\mathbf{u} = \sum_{i=1}^m u_i \mathbf{b}_i\}$$

The lattice Λ is an *integer lattice* when it is contained in $\mathbb{Z}^m$ instead of $\mathbb{R}^m$.

For prime modulus q , given a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and a column vector $\mathbf{u} \in \mathbb{Z}_q^n$, we define

$$\begin{aligned} \Lambda_q(\mathbf{A}) &= \{\mathbf{r} \in \mathbb{Z}^m \mid \exists \mathbf{v} \in \mathbb{Z}_q^n \text{ such that } \mathbf{A}^\top \mathbf{v} = \mathbf{r} \bmod q\} \\ \Lambda_q^\perp(\mathbf{A}) &= \{\mathbf{r} \in \mathbb{Z}^m \mid \mathbf{A}\mathbf{r} = \mathbf{0} \bmod q\} \\ \Lambda_q^{\mathbf{u}}(\mathbf{A}) &= \{\mathbf{r} \in \mathbb{Z}^m \mid \mathbf{A}\mathbf{r} = \mathbf{u} \bmod q\}. \end{aligned}$$

For any matrix $\mathbf{R} \in \mathbb{Z}_q^{n \times \ell}$, we define

$$\Lambda_q^{\mathbf{R}}(\mathbf{A}) = \{\mathbf{U} \in \mathbb{Z}_q^{m \times \ell} \mid \mathbf{A}\mathbf{U} = \mathbf{R} \bmod q\}$$

Observe that if $\mathbf{r}' \in \Lambda_q^{\mathbf{u}}(\mathbf{A})$ then $\Lambda_q^{\mathbf{u}}(\mathbf{A}) = \Lambda_q^\perp(\mathbf{A}) + \mathbf{r}'$ and $\Lambda_q^{\mathbf{u}}(\mathbf{A})$ is shift coset to $\Lambda_q^\perp(\mathbf{A})$.

Discrete Gaussian: [20] Let $\Lambda \subset \mathbb{Z}^m$ denote an integer lattice. The *Gaussian distribution* over $\mathbb{R}^m$, represented as $\rho_{\psi, \mathbf{c}}$ with centre $\mathbf{c} \in \mathbb{R}^m$ and a positive parameter $\psi \in \mathbb{R}$, is defined as

$$\rho_{\psi, \mathbf{c}}(\mathbf{x}) = e^{-\pi \frac{\|\mathbf{x} - \mathbf{c}\|^2}{\psi^2}}$$

for $\mathbf{x} \in \Lambda$ and $\rho_{\psi, \mathbf{c}}(\Lambda) = \sum_{\mathbf{x} \in \Lambda} \rho_{\psi, \mathbf{c}}(\mathbf{x})$ which is perpetually converging.

The *discrete Gaussian distribution* over Λ , represented as $\mathfrak{D}_{\Lambda, \psi, \mathbf{c}}$, is characterized by parameters ψ and $\mathbf{c}$, is specified as

$$\mathfrak{D}_{\Lambda, \psi, \mathbf{c}}(\mathbf{y}) = \frac{\rho_{\psi, \mathbf{c}}(\mathbf{y})}{\rho_{\psi, \mathbf{c}}(\Lambda)} \quad \text{for } \mathbf{y} \in \Lambda.$$

We abbreviate $\rho_{\psi, \mathbf{0}}$ and $\mathfrak{D}_{\Lambda, \psi, \mathbf{0}}$ as ρ_{ψ} and $\mathfrak{D}_{\Lambda, \psi}$ respectively. The distribution $\mathfrak{D}_{\Lambda, \psi, \mathbf{c}}$ is typically considered over the lattice $\Lambda = \Lambda_q^\perp(\mathbf{A})$ where $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ or over one of its cosets of the form $\Lambda = \mathbf{r}' + \Lambda_q^\perp(\mathbf{A})$ with $\mathbf{r}' \in \mathbb{Z}^m$.

For any matrix $\mathbf{M} \in \mathbb{Z}^{m \times n}$, we denote $\mathbf{M} \xleftarrow{\$} \mathfrak{D}_{\mathbb{Z}^{m \times n}, \psi}$ with parameter ψ to represent that each of the n columns of the matrix $\mathbf{M}$ is distributed according to the distribution $\mathfrak{D}_{\mathbb{Z}^m, \psi}$.

Definition 5 (Smoothing Parameter) [21] The smoothing parameter $\eta_\epsilon(\Lambda)$ is described as the least value of s such that $\rho_{1/\psi}(\Lambda^* - \{\mathbf{0}\}) \leq \epsilon$ for a lattice Λ of dimension n and positive real number $\epsilon > 0$ where $\rho_{1/\psi}$ is a Gaussian function centered at $\mathbf{0}$ with parameter $1/\psi > 0$ defined above.

For a lattice Λ with a basis $\mathbf{B} \in \mathbb{Z}^{n \times n}$, we define $\text{span}(\Lambda) = \text{span}(\mathbf{B}) = \{\mathbf{B}\mathbf{v} : \mathbf{v} \in \mathbb{R}^n\}$.

Lemma 1 (Bounding Gaussian Noise) [22] Consider a lattice $\Lambda \subset \mathbb{Z}^n$ with dimension n , centre $\mathbf{c} \in \text{span}(\Lambda)$. Let $\epsilon \in (0, 1) \cap \mathbb{R}$ and $\psi \in \eta_\epsilon(\Lambda)$ where $\eta_\epsilon(\Lambda)$ represents the smoothing parameter of Λ . Then

$$\Pr [\|\mathbf{x} - \mathbf{c}\| > \psi\sqrt{n}] \leq \left(\frac{1+\epsilon}{1-\epsilon}\right) \frac{1}{2^n} \quad \forall \mathbf{x} \xleftarrow{\$} \mathfrak{D}_{\Lambda, \psi, \mathbf{c}}.$$

Specifically, the following holds for any $\epsilon \in (0, \frac{1}{2})$, any $\psi \in \eta_\epsilon(\mathbb{Z})$ and any $t \in \omega(\sqrt{\log n})$ when integers are sampled

$$\Pr [|x - c| > \psi t] \leq \text{negl}(n) \quad \forall x \xleftarrow{\$} \mathfrak{D}_{\mathbb{Z}, \psi, c}.$$

Lemma 2 (Lemma 2.4 [23]) Let $\mathbf{Z}$ be sampled from $\mathfrak{D}_{\mathbb{Z}^{m \times m}, \sigma}$, then for any matrix $\mathbf{U}$ chosen uniformly at random from $\{-1, 1\}^{m \times m}$, the distribution of $\mathbf{Z}\mathbf{U}$ is statistically close to $\mathfrak{D}_{\mathbb{Z}^{m \times m}, \sigma\sqrt{m}}$.

Definition 6 (Learning with errors (LWE)) [24]: For a prime q , a positive real value α and $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n$, we define the oracles $\mathcal{O}_{\mathbf{s}}$ and $\mathcal{O}_{\S}$ as follows:

- Oracle $\mathcal{O}_{\mathbf{s}}$: Samples fresh and independently $\mathbf{a} \xleftarrow{\$} \mathbb{Z}_q^n$, $x \xleftarrow{\$} \mathfrak{D}_{\mathbb{Z}_q, \alpha q}$ and generates samples $(\mathbf{a}, \langle \mathbf{a}, \mathbf{s} \rangle + x) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$.
- Oracle $\mathcal{O}_{\S}$: Generates elements of $\mathbb{Z}_q^n \times \mathbb{Z}_q$ that are distributed uniformly at random.

Let $\mathcal{O}$ be an oracle which is either $\mathcal{O}_{\mathbf{s}}$ or $\mathcal{O}_{\S}$ for all invocations. The $\text{LWE}_{q, \alpha, n}$ problem distinguishes with non-negligible probability whether a given input $(\mathbf{a}, b) \in \mathbb{Z}_q^n \times \mathbb{Z}_q$ of the oracle $\mathcal{O}$ corresponds to $\mathcal{O}_{\mathbf{s}}$ or $\mathcal{O}_{\S}$ by utilizing the oracle $\mathcal{O}$.

Definition 7 (Decisional Approximate Shortest Vector Problem (GapSVP_γ) [25]) Given an arbitrary basis of an n dimensional lattice $\Lambda = \mathcal{L}(\mathbf{B})$, the GapSVP_γ problem is to determine whether $\lambda_1(\Lambda) \leq 1$ or $\lambda_1(\Lambda) > \gamma(n)$ where $\lambda_1(\Lambda) = \min_{\mathbf{v} \in \Lambda \setminus \{\mathbf{0}\}} \|\mathbf{v}\|$ and $\gamma(n) \geq 1$ is approximation factor.

Definition 8 (Shortest Independent Vector Problem (SIVP_γ) [25]) Given a basis $\mathbf{B}$ of an n -dimensional lattice $\Lambda = \mathcal{L}(\mathbf{B})$, the SIVP_γ problem asks for a set of linearly independent lattice vectors $\{\mathbf{s}_1, \dots, \mathbf{s}_n\} \subset \Lambda$ satisfying $\|\mathbf{s}_i\| \leq \gamma(n) \cdot \lambda_n(\Lambda)$ where λ_n denotes the n -th successive minimum of Λ . In other words, λ_n denotes the smallest k such that Λ contains n linearly independent vectors $\{\mathbf{s}_1, \dots, \mathbf{s}_n\}$ with norms bounded by k and $\gamma(n) \geq 1$ denotes the approximation factor.

Lemma 3 [24] Consider a positive integer n , a prime number $q = q(n)$ and a real number $\alpha = \alpha(n) \in (0, 1)$ where $\alpha q > 2\sqrt{n}$. Suppose there is an algorithm capable of efficiently solving the $\text{LWE}_{q, \alpha, n}$ problem. Then a quantum algorithm can efficiently approximate the shortest independent vectors problem (SIVP_γ) and the decisional variant of the shortest vector problem (GapSVP_γ) in the ℓ_2 norm for approximation factor $\gamma(n) \geq 1$. This holds in the worst-case scenario with accuracy of $\mathcal{O}(n/\alpha)$ factors.

Lemma 4 [13],[6]: Following result holds for the two PPT algorithms *TrapdoorGen* and *SamplePreimage* presented below for any positive integer n , any prime $q = \text{poly}(n)$, any $m \geq 2n \log q$ and any $\sigma \geq L\omega(\sqrt{\log n})$ with $L = m^{2.5}$:

- $\text{TrapdoorGen}(n, q, m, \psi) \rightarrow (\mathbf{A}, \mathbf{T})$: On input the parameters n, m, q and a real number ψ , the algorithm outputs a random matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ that is statistically close to uniform along with a good basis $\mathbf{T} (\in \mathbb{Z}^{m \times m}) \subset \Lambda_q^\perp(\mathbf{A})$ with $\|\tilde{\mathbf{T}}\| \leq L$. The basis $\mathbf{T}$ which satisfying $\mathbf{A}\mathbf{T} = \mathbf{0} \bmod q$ serves as a trapdoor for the matrix $\mathbf{A}$. The basis $\mathbf{T}$ is “good” trapdoor basis of Λ means that the Gram-Schmidt orthogonalized column vectors of $\mathbf{T}$ are relatively short. The i -th column $\mathbf{t}_i$ of $\mathbf{T}$ satisfies $\|\mathbf{t}_i\| \leq \psi\sqrt{m}$.
- $\text{SamplePreimage}(n, q, \mathbf{A}, \mathbf{T}, \sigma, \mathbf{U}, \ell) \rightarrow \mathbf{Z}$: This algorithm takes as input the parameters n, q , a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and a trapdoor $\mathbf{T} \in \mathbb{Z}^{m \times m}$ generated by *TrapdoorGen*, a matrix $\mathbf{U} \in \mathbb{Z}^{n \times \ell}$ chosen at random and $\sigma \geq L \cdot \omega(\sqrt{\log n})$ and generates a matrix $\mathbf{Z} \in \mathbb{Z}^{m \times \ell}$ satisfying $\mathbf{A}\mathbf{Z} = \mathbf{U}$. Furthermore, the following two distributions are statistically close

$$\left\{ (\mathbf{A}, \mathbf{Z}, \mathbf{U}) \left| \begin{array}{l} (\mathbf{A}, \mathbf{T}) \leftarrow \text{TrapdoorGen}(n, q, m, \psi), \\ \mathbf{Z} \leftarrow \text{SamplePreimage}(n, q, \mathbf{A}, \mathbf{T}, \sigma, \mathbf{U}, \ell) \end{array} \right. \right\} \approx_s \left\{ (\mathbf{A}, \mathbf{Z}, \mathbf{AZ}) \left| \begin{array}{l} \mathbf{A} \leftarrow \mathbb{Z}_q^{n \times m}, \mathbf{Z} \leftarrow \mathcal{D}_{\mathbb{Z}^{m \times \ell}, \sigma} \\ \|\mathbf{z}_i\| \leq \sigma\sqrt{m} \ \forall i \in [\ell] \text{ where } \mathbf{z}_i \\ \text{denotes the } i\text{-th column of } \mathbf{Z} \end{array} \right. \right\}$$

Definition 9 (Gadget Matrix [26]) For any modulus $q \geq 2$ with $l = \lceil \log q \rceil$, $\omega = nl$, the gadget matrix is defined as $\mathbf{G} = \mathbf{I}_n \otimes \mathbf{g} \in \mathbb{Z}_q^{n \times \omega}$ where $\mathbf{g} = (1, 2, \dots, 2^{l-1}) \in \mathbb{Z}_q^l$ and $\mathbf{I}_n$ denotes the identity matrix of order n .

Moreover, the deterministic inversion (bit decomposition) function is defined as $\mathbf{G}^{-1} : \mathbb{Z}_q^{n \times \bar{m}} \rightarrow \{0, 1\}^{\omega \times \bar{m}}$ where $\bar{m} \in \mathbb{Z}$ that decomposes each element of a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times \bar{m}}$ into its bit representation satisfying $\mathbf{G}\mathbf{G}^{-1}(\mathbf{A}) = \mathbf{A}$ for any matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times \bar{m}}$.

2.3 ALS-IPFE [2]

Below, we go over a modified variation of ALS-IPFE [2] proposed by Abdalla et al. [6]. The distribution with a complex parameter configuration is used to select the secret matrix $\mathbf{Z}$ in the original ALS-IPFE of Agrawal et al. [2]. For message space $\mathcal{X} = \{0, 1, \dots, X(\lambda) - 1\}^\ell$ and the decryption keys space $\mathcal{Y} = \{0, 1, \dots, Y(\lambda) - 1\}^\ell$ where λ is the security parameter and ℓ represents the size of both the message vector and the decryption key vector, the security proof presented in [6] allows the selection of $\mathbf{Z}$ from discrete Gaussian distribution. Furthermore, it is assumed that the inner product value falls inside the range $\{0, 1, \dots, K - 1\}$ i.e., $\langle \mathbf{x}, \mathbf{y} \rangle < K \forall \mathbf{x} \in \mathcal{X}$ and $\mathbf{y} \in \mathcal{Y}$ where $K = \ell XY$, $X = X(\lambda)$, $Y = Y(\lambda)$.

– **Setup**($1^\lambda, 1^\ell$) $\rightarrow$ (Mpk, Msk): On input λ and the size of input vector ℓ , the trusted authority first chooses the parameters (as given below in parameter setting) and runs the setup algorithm (see Algorithm 1) to produce the master public key Mpk = (pp, $\mathbf{A}_{\text{ALS}}$, $\mathbf{D}_{\text{ALS}}$) and master secret key Msk = $\mathbf{Z}_{\text{ALS}}$. Msk is kept secret and Mpk is made public.

Algorithm 1 Setup Algorithm

Input : λ ;

Output : Mpk = (pp, $\mathbf{A}_{\text{ALS}}$, $\mathbf{D}_{\text{ALS}}$) with pp = ($n, m, \ell, X, Y, K, q, \psi, \alpha, \sigma$); Msk = $\mathbf{Z}_{\text{ALS}}$;

- 1: $n = n(\lambda) \in \mathbb{Z}, m = 2n \log q \in \mathbb{Z}$;
 - 2: $X = X(\lambda) \in \mathbb{Z}, Y = Y(\lambda) \in \mathbb{Z}, K = \ell XY \in \mathbb{Z}$;
 - 3: $q = q(\lambda) \in \mathbb{Z}, \psi = \psi(\lambda) \in \mathbb{R}, \alpha = \alpha(\lambda) \in \mathbb{R}, \sigma = \sigma(\lambda) \in \mathbb{R}$;
 - 4: $\mathbf{A}_{\text{ALS}} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}, \mathbf{Z}_{\text{ALS}} \xleftarrow{\$} \mathcal{D}_{\mathbb{Z}^{m \times \ell}, \sigma}$;
 - 5: $\mathbf{D}_{\text{ALS}} = \mathbf{A}_{\text{ALS}} \mathbf{Z}_{\text{ALS}} \in \mathbb{Z}_q^{n \times \ell}$;
 - 6: **return** (pp = ($n, m, \ell, X, Y, K, q, \psi, \alpha, \sigma$), Mpk = (pp, $\mathbf{A}_{\text{ALS}}$, $\mathbf{D}_{\text{ALS}}$), Msk = $\mathbf{Z}_{\text{ALS}}$)
-

– **KeyExtract**(Mpk, Msk, $\mathbf{y}$) $\rightarrow$ $\text{Dk}_{\mathbf{y}}$: The trusted authority takes Mpk, Msk and $\mathbf{y} \in \mathcal{Y}$ to generate the decryption key $\text{Dk}_{\mathbf{y}} = (\mathbf{k}_1, \mathbf{k}_2)$ by running Algorithm 2 below. The trusted authority sends $\text{Dk}_{\mathbf{y}}$ to the decryptor through a secure channel.

Algorithm 2 Key Extraction Algorithm

Input : Mpk = (pp, $\mathbf{A}_{\text{ALS}}$, $\mathbf{D}_{\text{ALS}}$); Msk = $\mathbf{Z}_{\text{ALS}}$; $\mathbf{y}$;

Output : $\text{Dk}_{\mathbf{y}} = (\mathbf{k}_1, \mathbf{k}_2)$;

- 1: $\mathbf{k}_1 = \mathbf{y}^\top, \mathbf{k}_2 = \mathbf{y}^\top \mathbf{Z}_{\text{ALS}}^\top$;
 - 2: **return** ($\mathbf{k}_1, \mathbf{k}_2$)
-

– **Encrypt**(Mpk, $\mathbf{x}$) $\rightarrow$ $\text{Ct}_{\mathbf{x}}$: The encryptor intakes Mpk and a message vector $\mathbf{x} \in \mathcal{X}$ to compute the ciphertext $\text{Ct}_{\mathbf{x}}$ using Algorithm 3 presented below and sends $\text{Ct}_{\mathbf{x}}$ to the decryptor over a public channel.

Algorithm 3 Encryption Algorithm

Input : $\text{Mpk} = (\text{pp}, \mathbf{A}_{\text{ALS}}, \mathbf{D}_{\text{ALS}}); \mathbf{x};$

Output : $\text{Ct}_{\mathbf{x}} = (c_1, c_2);$

- 1: $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n, \mathbf{e}_1 \xleftarrow{\$} \mathcal{D}_{\mathbb{Z}^m, \psi}, \mathbf{e}_2 \xleftarrow{\$} \mathcal{D}_{\mathbb{Z}^\ell, \psi};$
 - 2: $\mathbf{c}_1 = \mathbf{A}_{\text{ALS}}^\top \mathbf{s} + \mathbf{e}_1 \in \mathbb{Z}_q^m;$
 - 3: $\mathbf{c}_2 = \mathbf{D}_{\text{ALS}}^\top \mathbf{s} + \lfloor \frac{q}{K} \rfloor \mathbf{x} + \mathbf{e}_2 \in \mathbb{Z}_q^\ell;$
 - 4: **return** (c_1, c_2)
-

– $\text{Decrypt}(\text{Mpk}, \text{Dk}_{\mathbf{y}}, \text{Ct}_{\mathbf{x}}) \rightarrow \mu \vee \perp$: On receiving the ciphertext $\text{Ct}_{\mathbf{x}}$ from an encryptor, a decryptor uses Mpk and its own secret key $\text{Dk}_{\mathbf{y}}$ corresponding to its secret key vector $\mathbf{y} \in \mathcal{Y}$ and executes the decryption algorithm as displayed in Algorithm 4.

Algorithm 4 Decryption Algorithm

Input : $\text{Mpk} = (\text{pp}, \mathbf{A}_{\text{ALS}}, \mathbf{D}_{\text{ALS}}); \text{Dk}_{\mathbf{y}} = (\mathbf{k}_1, \mathbf{k}_2); \text{Ct}_{\mathbf{x}} = (c_1, c_2);$

Output : $\mu;$

- 1: $\mu' = \langle \mathbf{k}_1^\top, c_2 \rangle - \langle \mathbf{k}_2^\top, c_1 \rangle;$
 - 2: **for** $\mu = 0$ to $K - 1$ **do**
 - 3: **if** $(\lfloor \frac{q}{K} \rfloor \cdot \mu - \mu' < (\sigma \psi m \ell Y + Y \ell \psi))$ **then**
 - 4: **break**;
 - 5: **end if**
 - 6: **end for**
 - 7: **return** μ
-

Lemma 5 (Correctness of ALS-IPFE [2],[6]) *If $\psi, \sigma > \omega(\sqrt{\log n})$ and $q > 2\ell K \sqrt{\ell Y} \omega(\log^2 n)$, then the scheme ALS-IPFE is correct.*

Parameter Settings

The parameter configuration for the ALS-IPFE which relies on the hardness of $\text{LWE}_{q, \alpha, n}$ is presented as $m \geq 2n \log q, \psi \geq \omega(\sqrt{\log n}), \sigma \geq \omega(\sqrt{\log n}), \alpha q \geq 2\sqrt{n}, \alpha \in (0, 1) \cap \mathbb{R}, \alpha \leq \frac{\psi}{2C_1 q (\sqrt{m} + \sqrt{n} + \sqrt{\ell})}$ where C_1 is a constant, $q \geq 2\ell K \sqrt{\ell Y} \omega(\log^2 n)$ with $K = \ell XY$ and αq represents the standard deviation of the noise distribution in the underlying LWE problem.

Lemma 6 (Security [2],[6]) *Let $q \geq 2\ell K \sqrt{\ell Y} \omega(\log^2 n)$ is prime, $m \geq 2n \log q, \psi \geq \omega(\sqrt{\log n}), \sigma \geq \omega(\sqrt{\log n})$, and $\alpha \leq \frac{\psi}{2C_1 q (\sqrt{m} + \sqrt{n} + \sqrt{\ell})}$ where C_1 is a constant. Then the scheme ALS-IPFE is considered Adp-IND (see Definition 4) secure based on the assumption of the hardness of $\text{LWE}_{q, \alpha, n}$ (see Definition 6).*

2.4 ulPFE Scheme of Dutta et al.[1]

In this section we discuss the *unbounded inner product functional encryption* scheme ulPFE of Dutta et al. [1] below. The co-ordinates of the message vectors are from $\mathcal{X} = \{0, 1, \dots, X(\lambda) - 1\}$ and those of the secret key vectors are from $\mathcal{Y} = \{0, 1, \dots, Y(\lambda) - 1\}$. The length of the message vector and the decryption key vector are less than $\ell_{\max}$. The inner product of a message vector $\mathbf{x} \in \mathcal{X}^{\ell_{\max}}$ and the associated decryption key vector $\mathbf{y} \in \mathcal{Y}^{\ell_{\max}}$ lies between $\{0, 1, \dots, K - 1\}$ i.e. $\langle \mathbf{x}, \mathbf{y} \rangle < K$, where $K = XY\ell_{\max}$.

– **ulPFE.Setup**(1^λ) $\rightarrow$ (Mpk, Msk): On input a security parameter λ , a trusted authority executes the steps as described in Algorithm 5 below choosing the parameters as given in parameter settings below and generates master public-key Mpk and master secret-key Msk. This algorithm makes use of the PPT algorithm **TrapdoorGen**(n, q, m, ψ) $\rightarrow$ ($\mathbf{A}, \mathbf{T}$) which generates a random matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ along with its trapdoor basis $\mathbf{T} (\in \mathbb{Z}^{m \times m}) \subseteq \Lambda_q^\perp(\mathbf{A})$ such that $\mathbf{AT} = \mathbf{0} \pmod{q}$, $\|\tilde{\mathbf{T}}\| \leq L$ with $L = m^{2.5}$, $\|\mathbf{t}_i\| \leq \psi\sqrt{m}$, $\mathbf{t}_i$ being the i -th column of $\mathbf{T}$ (see Lemma 4). Mpk is published, Msk is kept secret to the trusted authority.

Algorithm 5 ulPFE.Setup(1^λ) $\rightarrow$ (Mpk, Msk)

Input : λ ;

Output : Mpk = (pp, $\mathbf{A}$) with pp = ($n, q, m, \ell_{\max}, X, Y, K, \psi, \alpha, \sigma, \mathcal{H}$); Msk = $\mathbf{T}$;

- 1: $n = n(\lambda) \in \mathbb{Z}, q = q(\lambda) \in \mathbb{Z}, m = 2n \log_2 q \in \mathbb{Z}$;
 - 2: $\ell_{\max} = \ell_{\max}(\lambda) \in \mathbb{Z}, X = X(\lambda) \in \mathbb{Z}, Y = Y(\lambda) \in \mathbb{Z}, K = XY\ell_{\max} \in \mathbb{Z}$;
 - 3: $\psi = \psi(\lambda) \in \mathbb{R}, \alpha = \alpha(\lambda) \in \mathbb{R}, \sigma = \sigma(\lambda) \in \mathbb{R}$;
 - 4: Choose a hash function $\mathcal{H} : \mathbb{N} \rightarrow \mathbb{Z}_q^n$ which is modelled as random oracle;
 - 5: ($\mathbf{A}, \mathbf{T}$) $\leftarrow$ **TrapdoorGen**(n, q, m, ψ); $\mathbf{A} \in \mathbb{Z}_q^{n \times m}, \mathbf{T} \in \mathbb{Z}^{m \times m}$;
 - 6: **return** (pp = ($n, q, m, \ell_{\max}, X, Y, K, \psi, \alpha, \sigma, \mathcal{H}$), Mpk = (pp, $\mathbf{A}$), Msk = $\mathbf{T}$)
-

– **ulPFE.KeyExtract**(Mpk, Msk, $\mathbf{y}$) $\rightarrow$ $\text{Dk}_{\mathbf{y}}$: On input the Mpk, Msk and a secret key vector $\mathbf{y} = (y_i)_{i \in I_{\mathbf{y}}} \in \mathcal{Y}^{\ell'}$ received from a user, the trusted authority performs the steps as shown in Algorithm 6 below where $\ell' = |I_{\mathbf{y}}|$, $I_{\mathbf{y}}$ being the index set of $\mathbf{y}$. This algorithm makes use of the PPT algorithm **SamplePreimage**($n, q, \mathbf{A}, \mathbf{T}, \sigma, \mathbf{U}', \ell'$) $\rightarrow$ $\mathbf{Z}$ which generates a matrix $\mathbf{Z} \in \mathcal{D}_{\mathbb{Z}^{n \times \ell'}, \sigma}$ satisfying $\mathbf{AZ} = \mathbf{U}' \pmod{q}$.

Algorithm 6 $\text{ulPFE.KeyExtract}(\text{Mpk}, \text{Msk}, \mathbf{y}) \rightarrow \text{Dk}_{\mathbf{y}}$

Input : $\text{Mpk} = (\text{pp}, \mathbf{A})$ with $\text{pp} = (n, q, m, \ell_{\max}, X, Y, K, \psi, \alpha, \sigma, \mathcal{H})$; $\text{Msk} = \mathbf{T}$; $\mathbf{y} = (y_i)_{i \in I_{\mathbf{y}}}$ with $|I_{\mathbf{y}}| = \ell' \leq \ell_{\max}$;
Output : $\text{Dk}_{\mathbf{y}} = (\mathbf{y}^\top, \mathbf{y}^\top \mathbf{Z}^\top)$;
1: **for** $i = 1$ to ℓ' **do**
2: $\mathbf{u}'_i \leftarrow \mathcal{H}(I_{\mathbf{y}}[i])$;
3: **end for**
4: Set $\mathbf{U}' = [\mathbf{u}'_1 \| \mathbf{u}'_2 \| \dots \| \mathbf{u}'_{\ell'}] \in \mathbb{Z}_q^{n \times \ell'}$;
5: $\text{SamplePreimage}(n, q, \mathbf{A}, \mathbf{T}, \sigma, \mathbf{U}', \ell') \rightarrow \mathbf{Z} \in \mathfrak{D}_{\mathbb{Z}^{m \times \ell'}, \sigma}$ satisfying $\mathbf{AZ} = \mathbf{U}' \pmod{q}$;
6: **return** $\text{Dk}_{\mathbf{y}} = (\mathbf{y}^\top, \mathbf{y}^\top \mathbf{Z}^\top)$

– $\text{ulPFE.Encrypt}(\text{Mpk}, \mathbf{x}) \rightarrow \text{Ct}_{\mathbf{x}}$: The encryptor takes as input the Mpk and a message vector $\mathbf{x} = (x_i)_{i \in I_{\mathbf{x}}} \in \mathcal{X}^\ell$ where $\ell = |I_{\mathbf{x}}|$, $I_{\mathbf{x}} \subseteq \mathbb{N}$ is the non-empty index set of $\mathbf{x}$ and generates the ciphertext $\text{Ct}_{\mathbf{x}}$ following the steps presented in Algorithm 7 below and sends $\text{Ct}_{\mathbf{x}}$ over a public channel to the decryptor.

Algorithm 7 $\text{ulPFE.Encrypt}(\text{Mpk}, \mathbf{x}) \rightarrow \text{Ct}_{\mathbf{x}}$

Input : $\text{Mpk} = (\text{pp}, \mathbf{A})$ with $\text{pp} = (n, q, m, \ell_{\max}, X, Y, K, \psi, \alpha, \sigma, \mathcal{H})$; $\mathbf{x} = (x_i)_{i \in I_{\mathbf{x}}}$ with $|I_{\mathbf{x}}| = \ell \leq \ell_{\max}$;
Output : $\text{Ct}_{\mathbf{x}} = (c_1, c_2)$;
1: **for** $i = 1$ to ℓ **do**
2: $\mathbf{u}_i \leftarrow \mathcal{H}(I_{\mathbf{x}}[i])$;
3: **end for**
4: Set $\mathbf{U} = [\mathbf{u}_1 \| \mathbf{u}_2 \| \dots \| \mathbf{u}_\ell] \in \mathbb{Z}_q^{n \times \ell}$;
5: $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n, \mathbf{e}_1 \xleftarrow{\$} \mathfrak{D}_{\mathbb{Z}^m, \psi}, \mathbf{e}_2 \xleftarrow{\$} \mathfrak{D}_{\mathbb{Z}^\ell, \psi}$;
6: $\mathbf{c}_1 = \mathbf{A}^\top \mathbf{s} + \mathbf{e}_1 \in \mathbb{Z}_q^m$;
7: $\mathbf{c}_2 = \mathbf{U}^\top \mathbf{s} + \lfloor \frac{q}{K} \rfloor \mathbf{x} + \mathbf{e}_2 \in \mathbb{Z}_q^\ell$;
8: **return** $\text{Ct}_{\mathbf{x}} = (\mathbf{c}_1, \mathbf{c}_2)$

– $\text{ulPFE.Decrypt}(\text{Mpk}, \text{Dk}_{\mathbf{y}}, \text{Ct}_{\mathbf{x}}) \rightarrow \mu \vee \perp$: On input the Mpk , a decryptor uses its decryption key $\text{Dk}_{\mathbf{y}}$ corresponding to its secret key vector $\mathbf{y} \in \mathcal{Y}^{\ell'}$ and a ciphertext $\text{Ct}_{\mathbf{x}}$, the decryptor executes the steps as in Algorithm 8 below.

Algorithm 8 $\text{uIPFE.Decrypt}(\text{Mpk}, \text{Dk}_y, \text{Ct}_x) \rightarrow \mu \text{ or } \perp$

Input : $\text{Mpk} = (\text{pp}, \mathbf{A})$ with $\text{pp} = (n, q, m, \ell_{\max}, X, Y, K, \psi, \alpha, \sigma, \mathcal{H})$; $\text{Dk}_y = (\mathbf{y}^\top, \mathbf{y}^\top \mathbf{Z}^\top)$; $\text{Ct}_x = (\mathbf{c}_1, \mathbf{c}_2)$;

Output : μ ;

```

1:  $\mu' = \mathbf{y}^\top \mathbf{c}_2 - \mathbf{y}^\top \mathbf{Z}^\top \mathbf{c}_1$ ;
2: for  $(\mu = 0 \text{ to } K - 1)$  do
3:   if  $(\lfloor \frac{q}{K} \rfloor \mu - \mu') < \sigma \psi m \ell_{\max} Y + Y \ell_{\max} \psi)$  then
4:     break;
5:   end if
6: end for
7: return  $\mu$ 

```

Parameter Settings

The parameter setting of the uIPFE of Dutta et al. [1] which is based on $\text{LWE}_{q,\alpha,n}$ is presented as $m \geq 2n \log q$, $L = m^{2.5}$, $\sigma \geq L\omega(\sqrt{\log n})$, $\psi \geq \omega(\sqrt{\log n})$, $\alpha q \geq 2\sqrt{n}$, $\alpha \in (0, 1) \cap \mathbb{R}$, $\alpha \leq \frac{\psi}{2C_1 q(\sqrt{m} + \sqrt{n} + \sqrt{\ell_{\max}})}$ where C_1 is a constant, $q \geq 2K(\sigma \psi m \ell_{\max} Y + Y \ell_{\max} \psi)$ where $K = \ell_{\max} XY$ and αq represents the standard deviation of LWE's noise distribution.

Lemma 7 (Correctness of uIPFE of Dutta et al. [1], [11]) *If $m \geq 2n \log q$, $\psi \geq \omega(\sqrt{\log n})$, $L = m^{2.5}$, $\sigma \geq L\omega(\sqrt{\log n})$ and $q \geq 2K(\sigma \psi m \ell_{\max} Y + Y \ell_{\max} \psi)$, then the above scheme uIPFE is correct.*

Lemma 8 (Security [1], [11]) *The scheme uIPFE discussed above is considered Adp-IND secure (see Definition 4) in random oracle model (ROM) under the hardness of $\text{LWE}_{q,\alpha,n}$ problem (see Definition 6) where $m \geq 2n \log q$, $L = m^{2.5}$, $\sigma \geq L\omega(\sqrt{\log n})$, $\psi \geq \omega(\sqrt{\log n})$, $\alpha q \geq 2\sqrt{n}$, $\alpha \leq \frac{\psi}{2C_1 q(\sqrt{m} + \sqrt{n} + \sqrt{\ell_{\max}})}$ and $q \geq 2K(\sigma \psi m \ell_{\max} Y + Y \ell_{\max} \psi)$.*

3 Implementation of the uIPFE of Dutta et al. [1]

3.1 Implementation Overview

The scheme $\text{uIPFE} = (\text{uIPFE.Setup}, \text{uIPFE.Extract}, \text{uIPFE.Encrypt}, \text{uIPFE.Decrypt})$ is a trapdoor-based scheme as the setup algorithm uIPFE.Setup (Algorithm 5) invokes the trapdoor generation algorithm TrapdoorGen and the key extraction algorithm uIPFE.KeyExtract (Algorithm 6) calls the Gaussian preimage sampling algorithm SamplePreimage . Thus the largest part of the running time of uIPFE depends on the execution time of TrapdoorGen and SamplePreimage . We provide below the details of an efficient Gaussian sampler RandomSample which serves as a building block of both the algorithms TrapdoorGen and SamplePreimage in our implementation. In both algorithms uIPFE.KeyExtract (see Algorithm 6) and uIPFE.Encrypt (Algorithm 7), we instantiate SHAKE-256 as a hash function, which relies on the Keccak permutation.

- The procedure **RandomSample** described in Algorithm 9 samples a matrix $\mathbf{R}$ such that each entry of the matrix $\mathbf{R}$ is sampled according to the Gaussian distribution. It takes parameter ψ of the discrete Gaussian distribution and dimensions ℓ, m of the matrix $\mathbf{R}$ as input, executes the procedure **Sample** described in Algorithm 10 as a subroutine to sample each entry of $\mathbf{R}$ and finally outputs the matrix $\mathbf{R}$ of order $\ell \times m$ from discrete Gaussian distribution $\mathfrak{D}_{\mathbb{Z}^{\ell \times m}, \psi}$.

Algorithm 9 $\text{RandomSample}(\ell, m, \psi) \rightarrow \mathbf{R}$

Input : ℓ, m, ψ ;
Output : $\mathbf{R} = (r_{i,j})_{i \in [\ell], j \in [m]} \in \mathbb{Z}^{\ell \times m}$;
1: **for** ($i = 1$ to ℓ) **do**
2: **for** ($j = 1$ to m) **do**
3: $r_{i,j} \leftarrow \text{Sample}(1.0, \psi, 0.0)$;
4: **end for**
5: **end for**
6: **return** $\mathbf{R}$

Algorithm 10 $\text{Sample}(t, \psi, c) \rightarrow x$

Input : $t \in \text{FP}_{m_1}, \psi \in \text{FP}_{m_1}, c \in \text{FP}_{m_1}$;
Output : $x \in \mathbb{Z}$;
1: $h = -\pi/\psi^2 \in \text{FP}_{m_1}$;
2: **do**
3: $x \xleftarrow{\$} [c - t\psi, c + t\psi] \cap \mathbb{Z}$;
4: $r \xleftarrow{\$} [0, 1) \subset \text{FP}_{m_1}$
5: $p = e^{h(x-c)^2} \in \text{FP}_{m_1}$
6: **while** ($r < p$);
7: **return** x

- The subprocedure **Sample** adapts the rejection sampling algorithm for Discrete Gaussian on $\mathbb{Z}$ of [17] that takes tail-cut t , parameter ψ of discrete Gaussian distribution and centre c as input and outputs an integer x sampled from $\mathfrak{D}_{\mathbb{Z}, \psi}$ where $t \in \text{FP}_{m_1}, \psi \in \text{FP}_{m_1}$ and $c (= 0) \in \text{FP}_{m_1}$. Here, FP_{m_1} denotes the set of floating point numbers with mantissa size m_1 bits. A floating point number $f \in \text{FP}_{m_1}$ can be expressed as a triplet $f = (f_1, f_2, f_3)$ which represents the real number $f = (-1)^{f_1} \cdot 2^{f_2 - m_1} \cdot f_3 \in \mathbb{R}$ where $f_1 \in \{0, 1\}$, $f_2 \in \mathbb{Z}$ and $f_3 \in \mathbb{N}_{2^{m_1} - 1}$. Floating-point numbers have explicitly determined precisions based on their data types such as 32-bit single-precision and 64-bit double-precision which offer 23-bit and 52-bit mantissa respectively. We have used Gaussian sampler that samples over $\mathbb{Z}$ as this sampler can produce samples from $\mathbb{Z}$ in a constant time regardless of the output value, center or Gaussian parameter. The running time of this sampler of discrete Gaussian over $\mathbb{Z}$ depend on neither the input values nor on the output values. One can replace this sampler with any other sampler if required.
- The trapdoor generation algorithm **TrapdoorGen** presented in Algorithm 11 generates a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$ and its corresponding trapdoor $\mathbf{T} \in \mathbb{Z}^{m \times m}$ on input n, q, m and parameter ψ . It sets $l, \omega, \bar{m}$ as shown in the Algorithm 11, samples a matrix $\bar{\mathbf{A}}$ randomly from $\mathbb{Z}_q^{n \times \bar{m}}$. We set the tag $\mathbf{H} \in \mathbb{Z}_q^{n \times n}$ as an identity matrix of order n . The procedure **RandomSample** is invoked to generate a matrix $\mathbf{T}_\mathbf{A} \in \mathbb{Z}^{\bar{m} \times \omega}$ sampled from the discrete Gaussian distribution $\mathfrak{D}_{\mathbb{Z}^{\bar{m} \times \omega}, \psi}$. It then generates the gadget matrix $\mathbf{G} \in \mathbb{Z}_q^{n \times \omega}$ by computing tensor product of identity matrix $\mathbf{I}_n$ of order n and the gadget vector $\mathbf{g} = (1, 2, 2^2, \dots, 2^{l-1}) \in \mathbb{Z}_q^l$. Finally, it outputs the matrix

$$\mathbf{A} = [\overline{\mathbf{A}} \parallel \mathbf{H}\mathbf{G} - \overline{\mathbf{A}}\mathbf{T}_{\mathbf{A}}] \in \mathbb{Z}_q^{n \times m} \text{ and its trapdoor } \mathbf{T} = \begin{bmatrix} \mathbf{I}_{\overline{m}} + \mathbf{T}_{\mathbf{A}}\mathbf{W} & \mathbf{T}_{\mathbf{A}}\mathbf{S} \\ \mathbf{W} & \mathbf{S} \end{bmatrix} \in \mathbb{Z}^{m \times m}$$

where $\mathbf{W} = -\mathbf{G}^{-1}(\mathbf{A}) \in \mathbb{Z}^{\omega \times \overline{m}}$ and $\mathbf{S} \in \mathbb{Z}^{\omega \times \omega}$ is a basis for $\Lambda^\perp(\mathbf{G})$ as shown in the Algorithm 11. This procedure need to run a procedure `GetBit` described in Algorithm 12 for generating the matrix $\mathbf{S} \in \mathbb{Z}^{\omega \times \omega}$.

Algorithm 11 `TrapdoorGen` $(n, q, m, \psi) \rightarrow (\mathbf{A}, \mathbf{T})$

Input : $n \in \mathbb{Z}, m \in \mathbb{Z}, q \in \mathbb{Z}, \psi \in \mathbb{R}$;

Output : $\mathbf{A} \in \mathbb{Z}_q^{n \times m}, \mathbf{T} \in \mathbb{Z}^{m \times m}$;

1: $l = \lceil \log_2(q) \rceil \in \mathbb{Z}, \omega = n.l \in \mathbb{Z}, \overline{m} = (m - \omega) \in \mathbb{Z}$;

2: $\overline{\mathbf{A}} \xleftarrow{\$} \mathbb{Z}_q^{n \times \overline{m}}, \mathbf{H} = \mathbf{I}_n \in \mathbb{Z}_q^{n \times n}$;

3: $\mathbf{T}_{\mathbf{A}} \leftarrow \text{RandomSample}(\overline{m}, \omega, \psi); \mathbf{T}_{\mathbf{A}} \in \mathbb{Z}^{\overline{m} \times \omega}$;

4: $\mathbf{g} = (1, 2, 2^2, \dots, 2^{l-1}) \in \mathbb{Z}_q^l$;

5: $\mathbf{G} = \mathbf{I}_n \otimes \mathbf{g} \in \mathbb{Z}_q^{n \times \omega}$;

6: $\mathbf{A} = [\overline{\mathbf{A}} \parallel \mathbf{H} \cdot \mathbf{G} - \overline{\mathbf{A}} \cdot \mathbf{T}_{\mathbf{A}}] \in \mathbb{Z}_q^{m \times n}$;

7: **for** $(i = 1 \text{ to } l)$ **do**

8: $q_{(i)} = \text{GetBit}(q, i)$;

9: **end for**

$$10: \text{Set } \mathbf{S}_l = \begin{bmatrix} 2 & 0 & 0 & \dots & 0 & q_{(1)} \\ -1 & 2 & 0 & \dots & 0 & q_{(2)} \\ 0 & -1 & 2 & \dots & 0 & q_{(3)} \\ \vdots & \vdots & \vdots & \dots & \vdots & \vdots \\ 0 & 0 & 0 & \dots & -1 & q_{(l)} \end{bmatrix} \in \mathbb{Z}^{l \times l}; \mathbf{S} = \begin{bmatrix} \mathbf{S}_l & \mathbf{O}_l & \dots & \mathbf{O}_l \\ \mathbf{O}_l & \mathbf{S}_l & \dots & \mathbf{O}_l \\ \vdots & \vdots & \dots & \vdots \\ \mathbf{O}_l & \mathbf{O}_l & \dots & \mathbf{S}_l \end{bmatrix} \in \mathbb{Z}^{\omega \times \omega};$$

11: $\mathbf{W} = -\mathbf{G}^{-1}(\overline{\mathbf{A}}) \in \mathbb{Z}^{\omega \times \overline{m}}$;

$$12: \mathbf{T} = \begin{bmatrix} \mathbf{I}_{\overline{m}} & \mathbf{T}_{\mathbf{A}} \\ \mathbf{O} & \mathbf{I}_{\omega} \end{bmatrix} \cdot \begin{bmatrix} \mathbf{I}_{\overline{m}} & \mathbf{O} \\ \mathbf{W} & \mathbf{S} \end{bmatrix} = \begin{bmatrix} \mathbf{I}_{\overline{m}} + \mathbf{T}_{\mathbf{A}}\mathbf{W} & \mathbf{T}_{\mathbf{A}}\mathbf{S} \\ \mathbf{W} & \mathbf{S} \end{bmatrix} \in \mathbb{Z}^{m \times m};$$

13: **return** $(\mathbf{A}, \mathbf{T})$

- The procedure `GetBit` takes two integers q and i and returns the i -th bit $q_{(i)}$ of the given integer q . We denote ‘ a RIGHTSHIFT b ’ to represent the right-shifting of an integer ‘ a ’, ‘ b ’ bits which is equivalent to dividing a with 2^b and ‘ a AND b ’ represents the bit-by-bit and operator.

Algorithm 12 GetBit(q, i) $\rightarrow q_{(i)}$

Input : $q \in \mathbb{Z}, i \in \mathbb{Z}$;

Output : $q_{(i)} \in \mathbb{Z}$;

1: $q_{(i)} \leftarrow ((q \text{ RIGHTSHIFT } i) \text{ AND } 1)$;

2: **return** $q_{(i)}$

- The Gaussian preimage sampling algorithm **SamplePreimage** provided in algorithm 13 takes as input n, q , a matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, its trapdoor $\mathbf{T} \in \mathbb{Z}^{m \times m}$, Gaussian parameter $\sigma \in \mathbb{R}$, a matrix $\mathbf{U} \in \mathbb{Z}_q^{n \times \ell'}$ and number of columns ℓ' of $\mathbf{U}$ and generates a matrix $\mathbf{Z} \in \mathbb{Z}^{m \times \ell'}$ such that $\mathbf{AZ} = \mathbf{U}$.

Algorithm 13 SamplePreimage ($n, q, \mathbf{A}, \mathbf{T}, \sigma, \mathbf{U}, \ell'$) $\rightarrow \mathbf{Z}$

Input : $n \in \mathbb{Z}, q \in \mathbb{Z}, \mathbf{A} \in \mathbb{Z}_q^{n \times m}, \mathbf{T} \in \mathbb{Z}^{m \times m}, \sigma \in \mathbb{R}, \mathbf{U} \in \mathbb{Z}_q^{n \times \ell'}, \ell' \in \mathbb{Z}$;

Output : $\mathbf{Z} \in \mathbb{Z}^{m \times \ell'}$

1: $l = \log_2 q$; Choose $s \in \mathbb{R}$ where s is discrete Gaussian parameter such that $s \geq 10\sqrt{\log_2(2l(1 + \frac{1}{\epsilon}))/\pi}$, $0 < \epsilon \leq \frac{1}{2}$;

2: Parse $\mathbf{T} = \begin{bmatrix} \mathbf{T}_1 & \mathbf{T}_2 \\ \mathbf{T}_3 & \mathbf{T}_4 \end{bmatrix}$ as $\begin{bmatrix} \mathbf{I}_{\overline{m}} + \mathbf{T}_A \mathbf{W} & \mathbf{T}_A \mathbf{S} \\ \mathbf{W} & \mathbf{S} \end{bmatrix}$ where $\mathbf{T}_1 = \mathbf{I}_{\overline{m}} + \mathbf{T}_A \mathbf{W} \in \mathbb{Z}^{\overline{m} \times \overline{m}}$,

$\mathbf{T}_2 = \mathbf{T}_A \mathbf{S} \in \mathbb{Z}^{\overline{m} \times \omega}$, $\mathbf{T}_3 = \mathbf{W} \in \mathbb{Z}^{\omega \times \overline{m}}$ and $\mathbf{T}_4 = \mathbf{S} \in \mathbb{Z}^{\omega \times \omega}$;

3: $\mathbf{T}_A = \mathbf{T}_2 \cdot \mathbf{T}_4^{-1} \in \mathbb{Z}^{\overline{m} \times \omega}$;

4: **for** ($i = 1$ to ℓ') **do**

5: $\mathbf{z}_i \leftarrow \text{SamplePre}(n, q, \mathbf{A}, \mathbf{T}_A, s, \sigma, \mathbf{u}_i)$; $\mathbf{z}_i \in \mathbb{Z}^n$; $\mathbf{u}_i$ is the i -th column of $\mathbf{U}$;

6: **end for** // This for loop can be executed parallelly;

7: Set $\mathbf{Z} = [\mathbf{z}_1 \| \mathbf{z}_2 \| \dots \| \mathbf{z}_{\ell'}]$; $\mathbf{Z} \in \mathcal{D}_{\mathbb{Z}^{m \times \ell'}, \sigma}$;

8: **return** $\mathbf{Z}$

- The algorithm employs the procedure **SamplePre** for sampling the columns $\{\mathbf{z}_i\}_{1 \leq i \leq \ell'}$ of the matrix $\mathbf{Z}$ which in turn executes **SamplePerturbation** and **GSampling** described in Algorithm 15 and Algorithm 16 respectively as subroutines.
- The procedure **SamplePerturbation** takes matrix $\mathbf{T}_A \in \mathbb{Z}^{\overline{m} \times \omega}$ along with the parameters σ and s as input and runs the procedures **RandomSample** (Algorithm 9) and **Sample** (Algorithm 10) as subroutines to output a perturbation vector $\mathbf{p} \in \mathbb{Z}^m$. The vector $\mathbf{p}$ is drawn from a centered discrete Gaussian distribution over $\mathbb{Z}^m$ with covariance $\mathbf{S} \in \mathbb{R}^{\overline{m} \times \overline{m}}$ as shown in Algorithm 15. Following the method proposed by Bert et al. [15], the covariance matrix is decomposed into blocks of varying dimensions at each iteration, with the center updated correspondingly to iteratively sample the individual perturbation components $p_i \in \mathbb{Z}$.
- On the other hand, the procedure **GSampling** on input n , a vector $\mathbf{v} \in \mathbb{Z}^n$, modulus q and α as variance invokes the procedures **SampleG** and **GetBit** described in Algorithm

Algorithm 14 SamplePre $(n, q, \mathbf{A}, \mathbf{T}_\mathbf{A}, s, \sigma, \mathbf{u}) \rightarrow \mathbf{z}$

Input : $n \in \mathbb{Z}, q \in \mathbb{Z}, \mathbf{A} \in \mathbb{Z}_q^{n \times m}, \mathbf{T}_\mathbf{A} \in \mathbb{Z}^{\overline{m} \times \omega}, s \in \mathbb{R}, \sigma \in \mathbb{R}, \mathbf{u} \in \mathbb{Z}_q^n$;

Output : $\mathbf{z} \in \mathbb{Z}^m$;

- 1: $l = \lceil \log_2 q \rceil$; $\omega = n.l$;
 - 2: $\mathbf{p} \leftarrow \text{SamplePerturbation}(n, q, \mathbf{T}_\mathbf{A}, s, \sigma)$; $\mathbf{p} \in \mathbb{Z}^m$;
 - 3: $\mathbf{v} = \mathbf{u} - \mathbf{A} \cdot \mathbf{p} \in \mathbb{Z}_q^n$;
 - 4: $\mathbf{z}' \leftarrow \text{GSampling}(n, \mathbf{v}, s, q)$; $\mathbf{z}' \in \mathbb{Z}^\omega$;
 - 5: $\mathbf{z} = \mathbf{p} + \begin{bmatrix} \mathbf{T}_\mathbf{A} \\ \mathbf{I}_\omega \end{bmatrix} \cdot \mathbf{z}'$; $\mathbf{I}_\omega$ is identity matrix of order ω ; $\mathbf{z} \in \mathbb{Z}^m$
 - 6: **return** $\mathbf{z}$
-

Algorithm 15 SamplePerturbation $(n, q, \mathbf{T}_\mathbf{A}, s, \sigma) \rightarrow \mathbf{p}$

Input : $n \in \mathbb{Z}, q \in \mathbb{Z}, \mathbf{T}_\mathbf{A} \in \mathbb{Z}^{\overline{m} \times \omega}, s \in \mathbb{R}, \sigma \in \mathbb{R}$;

Output : $\mathbf{p} \in \mathbb{Z}^m$;

- 1: $\omega = n \lceil \log_2 q \rceil \in \mathbb{Z}$; $\overline{m} = m - \omega \in \mathbb{Z}$;
 - 2: $\mathbf{r} \leftarrow \text{RandomSample}(\omega, 1, \sqrt{\sigma^2 - s^2})$; $\mathbf{r} \in \mathbb{Z}^\omega$;
 - 3: $\mathbf{c} = \frac{s^2}{\sigma^2 - s^2} \cdot \mathbf{T}_\mathbf{A} \cdot \mathbf{r} \in \mathbb{R}^{\overline{m}}$;
 - 4: $\mathbf{S} = \sigma^2 \cdot \mathbf{I}_{\overline{m}} - (s^{-2} - \sigma^{-2})^{-1} \cdot \mathbf{T}_\mathbf{A} \cdot \mathbf{T}_\mathbf{A}^\top \in \mathbb{R}^{\overline{m} \times \overline{m}}$;
 - 5: **for** ($i = \overline{m}$ to 1) **do**
 - 6: Split matrix $\mathbf{S} = \begin{bmatrix} \mathbf{A} & \mathbf{B} \\ \mathbf{B}^\top & f \end{bmatrix}$ into block matrices;
 - 7: where $\mathbf{A} \in \mathbb{R}^{i \times i}$, $\mathbf{B} \in \mathbb{R}^{i \times 1}$, $\mathbf{B}^\top \in \mathbb{R}^{1 \times i}$ and $f \in \mathbb{R}^{1 \times 1}$;
 - 8: $\mathbf{c} = (\mathbf{c}', c_i)$; where $\mathbf{c}' \in \mathbb{R}^{\overline{m}}$, $c_i \in \mathbb{R}$;
 - 9: $p_i \leftarrow \text{Sample}(1, \sqrt{f}, c_i)$ by running Algorithm 10; $p_i \in \mathbb{R}$;
 - 10: $\mathbf{c} = \mathbf{c}' + f^{-1} \cdot (p_i - c_i) \cdot \mathbf{B} \in \mathbb{R}^{\overline{m}}$;
 - 11: $\mathbf{S} = \mathbf{A} - f^{-1} \cdot \mathbf{B} \cdot \mathbf{B}^\top \in \mathbb{R}^{i \times i}$;
 - 12: **end for**
 - 13: $\mathbf{p} = (p_1, \dots, p_{\overline{m}}, \mathbf{r}) \in \mathbb{Z}^m$;
 - 14: **return** $\mathbf{p}$
-

17 and Algorithm 12 as subroutines and outputs a vector $\mathbf{z}' \in \mathbb{Z}^\omega$. To generate the vector $\mathbf{z}' \in \mathbb{Z}^\omega$ from the coset $\Lambda_q^\mathbf{v}(\mathbf{G})$ of the $\mathbf{G}$ -lattice $\Lambda_q(\mathbf{G})$, n column vectors are sampled independently from the lattice $\Lambda_q^\perp(\mathbf{g})$ and arranged one after another to form the vector $\mathbf{z}' \in \mathbb{Z}^\omega$.

- The procedure SampleG in turn calls three procedures SampleGPerturb, SampleD and GetBit presented in Algorithm 18, Algorithm 19 and Algorithm 12 as subroutines and outputs a vector $\mathbf{t} \in \mathbb{Z}^l$.

Algorithm 16 $\text{GSampling}(n, \mathbf{v}, \alpha, q) \rightarrow \mathbf{z}'$

Input : $n \in \mathbb{Z}, \mathbf{v} \in \mathbb{Z}^m, \alpha \in \mathbb{R}, q \in \mathbb{Z}$;**Output** : $\mathbf{z}' \in \mathbb{Z}^\omega$;

```
1:  $l = \lceil \log_2(q) \rceil \in \mathbb{Z}; \omega = n \cdot l \in \mathbb{Z}$ ;  
2:  $d_1 = \text{GetBit}(q, 1)/2 \in \mathbb{R}, s_1 = \sqrt{2(1 + 1/l) + 1} \in \mathbb{R}, h_1 = 0 \in \mathbb{R}$ ;  
3: for ( $i = 2$  to  $l$ ) do  
4:    $d_i = (d_{i-1} + \text{GetBit}(q, i))/2 \in \mathbb{R}$ ; //  $d_i$  is the  $i$ -th component of  $\mathbf{d} \in \mathbb{R}^l$   
5:    $s_i = \sqrt{2(1 + 1/(l-i))} \in \mathbb{R}$ ; //  $s_i$  is the  $i$ -th component of  $\mathbf{s} \in \mathbb{R}^l$   
6:    $h_i = \sqrt{2(1 - 1/\{l - (i-1)\})} \in \mathbb{R}$ ; //  $h_i$  is the  $i$ -th component of  $\mathbf{h} \in \mathbb{R}^l$   
7: end for  
8: for ( $i = 1$  to  $n$ ) do  
9:    $\mathbf{t} \leftarrow \text{SampleG}(q, v_i, \mathbf{s}, \mathbf{h}, \mathbf{d}, s)$ ; where  $\mathbf{t} \in \mathbb{Z}^n$ ;  
10:  for ( $j = 1$  to  $l$ ) do  
11:     $z'_{(i-1) \cdot l + j} = t_j$ ; //  $t_j$  is the  $j$ -th component of  $\mathbf{t} \in \mathbb{Z}^l$ ,  $z'_j$  is the  $j$ -th  
    component of  $\mathbf{z}' \in \mathbb{Z}^\omega$   
12:  end for  
13: end for // This for loop can be executed parallelly;  
14: return  $\mathbf{z}'$ 
```

Algorithm 17 $\text{SampleG}(q, u, \mathbf{s}, \mathbf{h}, \mathbf{d}, s) \rightarrow \mathbf{t}$

Input : $q \in \mathbb{Z}, u \in \mathbb{Z}, \mathbf{s} \in \mathbb{R}^l, \mathbf{h} \in \mathbb{R}^l, \mathbf{d} \in \mathbb{R}^l, s \in \mathbb{R}$;**Output** : $\mathbf{t} \in \mathbb{Z}^l$;

```
1:  $\psi' = s/3 \in \mathbb{R}, l = \lceil \log_2(q) \rceil \in \mathbb{Z}$ ;  
2:  $\mathbf{p} \leftarrow \text{SampleGPerturb}(q, \psi', \mathbf{s}, \mathbf{h})$ ; where  $\mathbf{p} \in \mathbb{Z}^l$ ;  
3:  $c_1 = (\text{GetBit}(u, 1) - p_1)/2 \in \mathbb{R}$ ;  
4: for ( $i = 2$  to  $l$ ) do  
5:    $c_i = (c_{i-1} + \text{GetBit}(u, i) - p_i)/2 \in \mathbb{R}$ ;  
6: end for  
7:  $\mathbf{w} \leftarrow \text{SampleD}(q, \psi', \mathbf{c}, \mathbf{d})$ ; where  $\mathbf{w} \in \mathbb{Z}^l$ ;  
8:  $t_1 = 2 \cdot w_1 + \text{GetBit}(q, 1) \cdot w_l + \text{GetBit}(u, 1) \in \mathbb{Z}$ ;  
9: for ( $i = 2$  to  $l-1$ ) do  
10:   $t_i = 2 \cdot w_i - w_{i-1} + \text{GetBit}(q, i) \cdot w_{l-1} + \text{GetBit}(u, i) \in \mathbb{Z}$ ; //  $t_i$  is the  $i$ -th component  
    of  $\mathbf{t} \in \mathbb{Z}^l$ ;  
11: end for  
12:  $t_l = \text{GetBit}(q, l) \cdot w_l - w_{l-1} + \text{GetBit}(u, l) \in \mathbb{Z}$ ;  
13: return  $\mathbf{t}$ 
```

- The procedure `SampleGPerturb` outputs a perturbation vector $\mathbf{p} \in \mathbb{Z}^m$ and procedure `SampleD` produces a vector $\mathbf{w} \in \mathbb{Z}^l$ using the rejection sampling algorithm `Sample` described as Algorithm 10.

Algorithm 18 $\text{SampleGPerturb}(q, \psi', \mathbf{s}, \mathbf{h}) \rightarrow \mathbf{p}$

Input : $q \in \mathbb{Z}, \psi' \in \mathbb{R}, \mathbf{s} \in \mathbb{R}^l, \mathbf{h} \in \mathbb{R}^l$;**Output :** $\mathbf{p} \in \mathbb{Z}^l$;

```
1:  $\beta = 0 \in \mathbb{R}, z_1'' = 0 \in \mathbb{Z}, l = \log_2(q)$ ;  
2: for ( $i = 1$  to  $l$ ) do  
3:    $c_i = \beta/s_i \in \mathbb{R}, \text{frac}(c_i) = c_i - \lfloor c_i \rfloor \in [0, 1), \psi'_i = \psi'/s_i \in \mathbb{R}$ ; //  $s_i$  is the  $i$ -th  
   component of  $\mathbf{s} \in \mathbb{R}^l$   
4:    $z_i'' \leftarrow \lfloor c_i \rfloor + \text{Sample}(1, \psi'_i, \text{frac}(c_i)) \in \mathbb{Z}$ ; using Algorithm 10; //  $z_i''$  is the  $i$ -th  
   component of  $\mathbf{z}'' \in \mathbb{Z}^l$   
5:    $\beta = -z_i'' \cdot h_i \in \mathbb{R}$ ; //  $h_i$  is the  $i$ -th component of  $\mathbf{h} \in \mathbb{R}^l$   
6: end for  
7:  $p_1 = 5 \cdot z_1'' + 2 \cdot z_2'' \in \mathbb{Z}$ ;  
8: for ( $i = 2$  to  $l - 1$ ) do  
9:    $p_i = 2(z_{i-1}'' + 2 \cdot z_i'' + z_{i+1}'')$ ; //  $p_i$  is the  $i$ -th component of  $\mathbf{p} \in \mathbb{Z}^l$   
10: end for  
11:  $p_l = 2(z_{l-1}'' + 2 \cdot z_l'')$ ;  
12: return  $\mathbf{p}$ 
```

Algorithm 19 $\text{SampleD}(q, \psi', \mathbf{c}, \mathbf{d}) \rightarrow \mathbf{w}$

Input : $q \in \mathbb{Z}, \psi' \in \mathbb{R}, \mathbf{c} \in \mathbb{R}^l, \mathbf{d} \in \mathbb{R}^l$;**Output :** $\mathbf{w} \in \mathbb{Z}^l$;

```
1:  $l = \log_2 q$ ;  
2:  $w_l \leftarrow \text{Sample}(1, \psi'/d_l, -c_l/d_l)$  using Algorithm 10; where  $w_l \in \mathbb{Z}$ ;  
3:  $\mathbf{c} = \mathbf{c} - w_l \cdot \mathbf{d} \in \mathbb{R}^l$ ;  
4: for ( $i = 1$  to  $l - 1$ ) do  
5:    $w_i \leftarrow \text{Sample}(1, \psi', -c_i)$ ; //  $w_i$  is the  $i$ -th component of  $\mathbf{w} \in \mathbb{Z}^l$   
6: end for  
7: return  $\mathbf{w}$ 
```

3.2 Parameter Choice

We employ a procedure **Param** for the choice of parameters for the uIPFE [1]. This procedure generates the parameters which satisfy the constraints given in parameter settings in subsection 2.3.

- The procedure **Param** on input $n, \ell_{\max}, \sigma, X, Y$ generates q, m, K, ψ, α by using the procedure **PrimeGen** as a subroutine.
- The procedure **PrimeGen** takes an integer a as input and generates a prime number p greater than a . It calls a procedure **isPrime** as subroutine.

Algorithm 20 $\text{Param}(n, \ell_{\max}, \sigma, X, Y) \rightarrow (q, m, K, \psi, \alpha)$

Input : $n \in \mathbb{Z}, \ell_{\max} \in \mathbb{Z}, \sigma \in \mathbb{R}, X \in \mathbb{Z}, Y \in \mathbb{Z};$ **Output :** $q \in \mathbb{Z}, m \in \mathbb{Z}, K \in \mathbb{Z}, \psi \in \mathbb{R}, \alpha \in \mathbb{R};$

```
1:  $a = 2 \in \mathbb{Z}, i = 0 \in \mathbb{Z}, j = 2 \in \mathbb{Z};$ 
2:  $K = XY\ell_{\max} \in \mathbb{Z}; \psi = \sqrt{n} \in \mathbb{R};$ 
3: do
4:    $l = \lceil \log_2 a \rceil \in \mathbb{Z}; m = 2 \cdot n \cdot l \in \mathbb{Z};$ 
5:    $\text{rhs} = 2K(\sigma\psi m\ell_{\max}Y + Y\ell_{\max}\psi) \in \mathbb{R};$ 
6:    $a = 2^j; j = j + 1;$ 
7: while  $(a < \text{rhs});$ 
8:  $q = \text{PrimeGen}(a) \in \mathbb{Z};$ 
9:  $l = \lceil \log_2 q \rceil \in \mathbb{Z}; m = 2 \cdot n \cdot l \in \mathbb{Z};$ 
10: do
11:    $\alpha = \frac{\psi}{2 \cdot q \cdot 10^{-i} \cdot (\sqrt{m} + \sqrt{n} + \sqrt{\ell_{\max}})} \in \mathbb{R};$ 
12:    $i = i + 1;$ 
13: while  $(\alpha \cdot q < 2 \cdot \sqrt{n});$ 
14: return  $q, m, K, \psi, \alpha$ 
```

Algorithm 21 $\text{PrimeGen}(a) \rightarrow p$

Input : $a \in \mathbb{Z};$ **Output :** $p \in \mathbb{Z};$

```
1:  $p = a + 1;$ 
2: while (1) do
3:   if  $(\text{isPrime}(p))$  then
4:     break;
5:   end if
6:    $p = p + 1;$ 
7: end while
8: return  $p$ 
```

Algorithm 22 $\text{isPrime}(p) \rightarrow 1 \vee 0$

Input : $p \in \mathbb{Z};$ **Output :** $1 \vee 0;$

```
1: for  $(i = 2 \text{ to } \sqrt{p})$  do
2:   if  $((p \text{ MODULO } i) = 0)$  then
3:     return 0
4:   break;
5: end if
6: end for
7: return 1
```

- The procedure isPrime takes an integer p and returns 1 if p is a prime integer and 0 otherwise. We denote ' $a \text{ MODULO } b$ ' to represent the modulo operator which yields the remainder when the integer a is divided by the integer b .

4 Our scheme uIPFE #1

This section presents our *unbounded inner product functional encryption* scheme uIPFE scheme #1. The co-ordinates of the message vectors are from $\mathcal{X} = \{0, 1, \dots, X(\lambda) - 1\}$ and those of the secret key vectors are from $\mathcal{Y} = \{0, 1, \dots, Y(\lambda) - 1\}$. The length of the message vector and the decryption key vector are less than $\ell_{\max}$. The inner product between the message vector $\mathbf{x} \in \mathcal{X}^{\ell_{\max}}$ and the associated decryption key vector $\mathbf{y} \in \mathcal{Y}^{\ell_{\max}}$ lies between $\{0, 1, \dots, K - 1\}$, that is, $\langle \mathbf{x}, \mathbf{y} \rangle < K$ where $K = XY\ell_{\max}$.

– $\text{uIPFE.Setup}(1^\lambda) \rightarrow (\text{Mpk}, \text{Msk})$: On input a security parameter λ , a trusted authority executes the steps as described in Algorithm 23 below choosing the parameters as in Table 2 and generates master public-key Mpk and master secret-key Msk . Mpk is published and Msk is kept secret to the trusted authority.

Algorithm 23 $\text{uIPFE.Setup}(1^\lambda) \rightarrow (\text{Mpk}, \text{Msk})$

Input : λ ;

Output : $\text{Mpk} = (\text{pp}, \mathbf{A}, \mathbf{W})$ with $\text{pp} = (n, q, m, \ell_{\max}, X, Y, K, \psi, \alpha, \sigma, \mathcal{H})$; $\text{Msk} = \mathbf{Z}$;

- 1: $n = n(\lambda) \in \mathbb{Z}, q = q(\lambda) \in \mathbb{Z}, m = 2n \log_2 q \in \mathbb{Z}$;
 - 2: $\ell_{\max} = \ell_{\max}(\lambda) \in \mathbb{Z}, X = X(\lambda) \in \mathbb{Z}, Y = Y(\lambda) \in \mathbb{Z}, K = XY\ell_{\max} \in \mathbb{Z}$;
 - 3: $\psi = \psi(\lambda) \in \mathbb{R}, \alpha = \alpha(\lambda) \in \mathbb{R}, \sigma = \sigma(\lambda) \in \mathbb{R}$;
 - 4: Choose a hash function $\mathcal{H} : \mathbb{N} \rightarrow \{-1, 1\}^{\ell_{\max}}$ which is modelled as random oracle;
 - 5: $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$;
 - 6: $\mathbf{Z} \leftarrow \text{RandomSample}(m, \ell_{\max}, \sigma)$; $\mathbf{Z} \in \mathcal{D}_{\mathbb{Z}^{m \times \ell_{\max}}, \sigma}$;
 - 7: Compute $\mathbf{W} = \mathbf{AZ} \pmod{q} \in \mathbb{Z}_q^{n \times \ell_{\max}}$;
 - 8: **return** $(\text{pp} = (n, q, m, \ell_{\max}, X, Y, K, \psi, \alpha, \sigma, \mathcal{H}), \text{Mpk} = (\text{pp}, \mathbf{A}, \mathbf{W}), \text{Msk} = \mathbf{Z})$
-

– $\text{uIPFE.KeyExtract}(\text{Mpk}, \text{Msk}, \mathbf{y}) \rightarrow \text{Dk}_{\mathbf{y}}$: On input the master public key Mpk , master secret key Msk and a secret key vector $\mathbf{y} = (y_i)_{i \in I_{\mathbf{y}}} \in \mathcal{Y}^{\ell'}$ received from a user, the trusted authority performs the steps as shown in Algorithm 24 below where $\ell' = |I_{\mathbf{y}}|$, $I_{\mathbf{y}}$ being the index set of $\mathbf{y}$. The decryption key $\text{Dk}_{\mathbf{y}}$ generated by this algorithm is sent to the user through a secure communication channel.

Algorithm 24 $\text{uIPFE.KeyExtract}(\text{Mpk}, \text{Msk}, \mathbf{y}) \rightarrow \text{Dk}_{\mathbf{y}}$

Input : $\text{Mpk} = (\text{pp}, \mathbf{A}, \mathbf{W})$ with $\text{pp} = (n, q, m, \ell_{\max}, X, Y, K, \psi, \alpha, \sigma, \mathcal{H})$; $\text{Msk} = \mathbf{Z}$; $\mathbf{y} = (y_i)_{i \in I_{\mathbf{y}}}$ with $|I_{\mathbf{y}}| = \ell' \leq \ell_{\max}$;

Output : $\text{Dk}_{\mathbf{y}} = (\mathbf{y}^\top, \mathbf{y}^\top \mathbf{U}'^\top \mathbf{Z}^\top)$;

- 1: **for** $i = 1$ to ℓ' **do**
 - 2: $\mathbf{u}'_i \leftarrow \mathcal{H}(I_{\mathbf{y}}[i])$;
 - 3: **end for**
 - 4: Set $\mathbf{U}' = [\mathbf{u}'_1 \| \mathbf{u}'_2 \| \dots \| \mathbf{u}'_{\ell'}] \in \{-1, 1\}^{\ell_{\max} \times \ell'}$;
 - 5: **return** $\text{Dk}_{\mathbf{y}} = (\mathbf{y}^\top, \mathbf{y}^\top \mathbf{U}'^\top \mathbf{Z}^\top)$
-

– $\text{uIPFE.Encrypt}(\text{Mpk}, \mathbf{x}) \rightarrow \text{Ct}_{\mathbf{x}}$: The encryptor takes as input the Mpk and a message vector $\mathbf{x} = (x_i)_{i \in I_{\mathbf{x}}} \in \mathcal{X}^\ell$ where $\ell = |I_{\mathbf{x}}|$, $I_{\mathbf{x}} \subseteq \mathbb{N}$ is the non-empty index set of $\mathbf{x}$ and generates the ciphertext $\text{Ct}_{\mathbf{x}}$ following the steps presented in Algorithm 25 below and sends $\text{Ct}_{\mathbf{x}}$ over a public channel to the decryptor.

Algorithm 25 $\text{uIPFE.Encrypt}(\text{Mpk}, \mathbf{x}) \rightarrow \text{Ct}_{\mathbf{x}}$

Input : $\text{Mpk} = (\text{pp}, \mathbf{A}, \mathbf{W})$ with $\text{pp} = (n, q, m, \ell_{\max}, X, Y, K, \psi, \alpha, \sigma, \mathcal{H})$; $\mathbf{x} = (x_i)_{i \in I_{\mathbf{x}}}$ with $|I_{\mathbf{x}}| = \ell \leq \ell_{\max}$;
Output : $\text{Ct}_{\mathbf{x}} = (c_1, c_2)$;
1: **for** $i = 1$ **to** ℓ **do**
2: $\mathbf{u}_i \leftarrow \mathcal{H}(I_{\mathbf{x}}[i])$;
3: **end for**
4: Set $\mathbf{U} = [\mathbf{u}_1 \| \mathbf{u}_2 \| \dots \| \mathbf{u}_{\ell}] \in \{-1, 1\}^{\ell_{\max} \times \ell}$;
5: $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n, \mathbf{e}_1 \xleftarrow{\$} \mathcal{D}_{\mathbb{Z}^m, \psi}, \mathbf{e}_2 \xleftarrow{\$} \mathcal{D}_{\mathbb{Z}^{\ell}, \psi}$;
6: $\mathbf{c}_1 = \mathbf{A}^{\top} \mathbf{s} + \mathbf{e}_1 \in \mathbb{Z}_q^m$;
7: $\mathbf{c}_2 = \mathbf{U}^{\top} \mathbf{W}^{\top} \mathbf{s} + \lfloor \frac{q}{K} \rfloor \mathbf{x} + \mathbf{e}_2 \in \mathbb{Z}_q^{\ell}$;
8: **return** $\text{Ct}_{\mathbf{x}} = (c_1, c_2)$

– $\text{uIPFE.Decrypt}(\text{Mpk}, \text{Dk}_{\mathbf{y}}, \text{Ct}_{\mathbf{x}}) \rightarrow \mu \vee \perp$: On input the Mpk , a decryptor uses its decryption key $\text{Dk}_{\mathbf{y}}$ corresponding to its secret key vector $\mathbf{y} \in \mathcal{Y}^{\ell'}$ and a ciphertext $\text{Ct}_{\mathbf{x}}$, the decryptor executes the steps as in Algorithm 26 below.

Algorithm 26 $\text{uIPFE.Decrypt}(\text{Mpk}, \text{Dk}_{\mathbf{y}}, \text{Ct}_{\mathbf{x}}) \rightarrow \mu \text{ or } \perp$

Input : $\text{Mpk} = (\text{pp}, \mathbf{A}, \mathbf{W})$ with $\text{pp} = (n, q, m, \ell_{\max}, X, Y, K, \psi, \alpha, \sigma, \mathcal{H})$; $\text{Dk}_{\mathbf{y}} = (\mathbf{y}^{\top}, \mathbf{y}^{\top} \mathbf{U}'^{\top} \mathbf{Z}^{\top})$, $\text{Ct}_{\mathbf{x}} = (c_1, c_2)$;
Output : $\mu = \langle \mathbf{x}, \mathbf{y} \rangle$;
1: $\mu' = \mathbf{y}^{\top} c_2 - \mathbf{y}^{\top} \mathbf{U}'^{\top} \mathbf{Z}^{\top} c_1$;
2: **for** $(\mu = 0 \text{ to } K - 1)$ **do**
3: **if** $(|\lfloor \frac{q}{K} \rfloor \mu - \mu'| < \sigma \psi m Y \ell_{\max}^2 + Y \ell_{\max} \psi)$ **then**
4: **break**;
5: **end if**
6: **end for**
7: **return** μ

Parameter Setting

The parameter setting of our scheme $\text{uIPFE} \#1$ which is based on $\text{LWE}_{q, \alpha, n}$ is presented below in Table 2 where αq represents the standard deviation of LWE 's noise distribution.

Correctness

Correctness of our scheme $\text{uIPFE} \#1 = (\text{Setup}, \text{KeyExtract}, \text{Encrypt}, \text{Decrypt})$ follows from the following observation. The decryption succeed if $I_{\mathbf{x}} = I_{\mathbf{y}}$ (i.e., $\ell = \ell' \leq \ell_{\max}$), we have

$$\begin{aligned} \mu' &= \mathbf{y}^{\top} c_2 - \mathbf{y}^{\top} \mathbf{U}'^{\top} \mathbf{Z}^{\top} c_1 \\ &= \mathbf{y}^{\top} (\mathbf{U}^{\top} \mathbf{W}^{\top} \mathbf{s} + \lfloor \frac{q}{K} \rfloor \mathbf{x} + \mathbf{e}_2) - \mathbf{y}^{\top} \mathbf{U}'^{\top} \mathbf{Z}^{\top} (\mathbf{A}^{\top} \mathbf{s} + \mathbf{e}_1) \end{aligned}$$

Table 2: Parameters for our uIPFE

Parameters	Constraints
m, n	$m \geq 2n \log q$
ψ, σ	$\psi \geq \omega(\sqrt{\log n}), \sigma \geq \omega(\sqrt{\log n})$
α, q, n	$\alpha q \geq 2\sqrt{n}$ where $\alpha \in (0, 1) \cap \mathbb{R}$
$\alpha, q, m, n, \ell_{\max}$	$\alpha \leq \frac{\psi}{2C_1 q(\sqrt{m} + \sqrt{n} + \sqrt{\ell_{\max}})}$ where C_1 is a constant
$q, \sigma, \psi, \ell_{\max}, X, Y$	$q \geq 2K(\sigma\psi m Y \ell_{\max}^2 + Y \ell_{\max} \psi)$ where $K = \ell_{\max} X Y$

$$= \left\lfloor \frac{q}{K} \right\rfloor \langle \mathbf{x}, \mathbf{y} \rangle + \underbrace{\mathbf{y}^\top \mathbf{e}_2 - \mathbf{y}^\top \mathbf{U}'^\top \mathbf{Z}^\top \mathbf{e}_1}_{\text{noise}}.$$

Decryption will give correct output if the magnitude of the error quantity $\text{noise} = \mathbf{y}^\top \mathbf{e}_2 - (\mathbf{y}^\top \mathbf{U}'^\top \mathbf{Z}^\top) \mathbf{e}_1$ remains small. Since $\mathbb{Z} \in \mathcal{D}_{\mathbb{Z}^{m \times \ell_{\max}}, \sigma}$, every column $\mathbf{z}_i$ of $\mathbf{Z} \in \mathbb{Z}^{m \times \ell_{\max}}$ satisfies $\|\mathbf{z}_i\| \leq \sigma\sqrt{m}$ yielding $\|\mathbf{Z}\| \leq \sigma\sqrt{m\ell_{\max}}$ and since $\mathbf{U}' \in \{-1, 1\}^{\ell_{\max} \times \ell}$, $\|\mathbf{U}'\| \leq \sqrt{\ell_{\max}\ell}$. Also $\|\mathbf{y}\| \leq Y\sqrt{\ell}$ as $\mathbf{y} \in \mathcal{Y}^{|I_{\mathbf{y}}|} = \{0, 1, \dots, Y-1\}^{|I_{\mathbf{y}}|}$ with $|I_{\mathbf{y}}| = \ell' = \ell$, $\mathbf{e}_1 \xleftarrow{\$} \mathcal{D}_{\mathbb{Z}^m, \psi}$ and $\mathbf{e}_2 \xleftarrow{\$} \mathcal{D}_{\mathbb{Z}^\ell, \psi}$ implies $\|\mathbf{e}_1\| \leq \sqrt{m}\psi$ and $\|\mathbf{e}_2\| \leq \sqrt{\ell}\psi$ which in turn implies that

$$\|(\mathbf{y}^\top \mathbf{U}'^\top \mathbf{Z}^\top) \mathbf{e}_1\| \leq \|\mathbf{y}\| \|\mathbf{U}'\| \|\mathbf{Z}\| \|\mathbf{e}_1\| \leq Y\sqrt{\ell} \sqrt{\ell_{\max}\ell} \sigma \sqrt{m\ell_{\max}} \psi \sqrt{m} \leq \sigma\psi m \ell Y \ell_{\max}$$

by choosing $\psi \geq 2C_1 \alpha q (\sqrt{m} + \sqrt{n} + \sqrt{\ell})$ where C_1 is a constant. Therefore, $|\text{noise}| \leq \sigma\psi m \ell Y \ell_{\max} + Y \ell \psi$ with overwhelming probability. To guarantee the decryption's accuracy, it is crucial to make sure that the amount of noise is smaller than $\frac{q}{2K}$ which can be achieved by setting $q \geq 2K(\sigma\psi m Y \ell_{\max}^2 + Y \ell_{\max} \psi)$ where $\ell_{\max}$ represents the maximum magnitude of the message vectors and the decryption key vectors.

Theorem 9 Assume that the modified variation of ALS-IPFE discussed in subsection 2.3 is *Adp-IND* secure according to Definition 4 depending upon the validity of the $\text{LWE}_{q, \alpha, n}$ assumption (see Definition 6) for the condition $q \geq 2K\sigma\psi m \sqrt{|I|}Y + 2KY|I|\psi$ where $n, m, K, X, Y, \sigma, \psi$ are public parameters as described above in parameter settings of subsection 2.3. Then our scheme uIPFE #1 = (uIPFE.Setup, uIPFE.KeyExtract, uIPFE.Encrypt, uIPFE.Decrypt) presented above is *Adp-IND* secure according to Definition 4 in the random oracle model (ROM) under the hardness of $\text{LWE}_{q, \alpha, n}$ problem where $m \geq 2n \log q, \sigma \geq \omega(\sqrt{\log n}), \psi \geq \omega(\sqrt{\log n}), \alpha q \geq 2\sqrt{n}, \alpha \leq \frac{\psi}{2C_1 q(\sqrt{m} + \sqrt{n} + \sqrt{\ell_{\max}})}$ and $q \geq 2K(\sigma\psi m Y \ell_{\max}^2 + Y \ell_{\max} \psi)$.

Proof To demonstrate the security of our uIPFE #1, we conduct a sequences of hybrid games involving a challenger $\mathcal{B}$ and a PPT adversary $\mathcal{A}$. Assume that $\text{Adv}_{\mathcal{A}, \text{Game}_i}(\lambda)$ represents $\mathcal{A}$'s advantage in Game i . We consider without loss of generality that when $\mathcal{A}$ makes the j -th decryption key query for the secret key on a vector $\mathbf{y}^{(j)} = (y_i^{(j)})_{i \in I}$, $\mathcal{A}$ initially makes

query to a random oracle (RO) $\mathcal{H}$ for the finite index set $I \subseteq \mathbb{N}$ where $|I| \leq \ell_{\max}$ with $\ell_{\max}$ denoting the maximum magnitude of message vectors and the secret key vectors.

Game 0: $\mathcal{B}$ produces $(\text{Mpk}, \text{Msk}) \leftarrow \text{ulPFE.Setup}(1^\lambda)$ with $\text{Mpk} = (\text{pp}, \mathbf{A}, \mathbf{W})$, $\text{Msk} = \mathbf{Z}$ and transmits Mpk to $\mathcal{A}$. We have, $\mathbf{A} \in \mathbb{Z}_q^{n \times m}$, $\mathbf{W} \in \mathbb{Z}_q^{n \times \ell_{\max}}$, $\mathbf{Z} \in \mathcal{D}_{\mathbb{Z}^m \times \ell_{\max}, \sigma}$, pp includes $m = m(\lambda) \in \mathbb{Z}, n = n(\lambda) \in \mathbb{Z}, X = X(\lambda) \in \mathbb{Z}_q, Y = Y(\lambda) \in \mathbb{Z}_q, \psi(\lambda), \alpha(\lambda), \sigma(\lambda) \in \mathbb{R}^+$ and $K = \ell_{\max}XY \in \mathbb{Z}$ is polynomially constrained in λ . After receiving pp , $\mathcal{A}$ makes a polynomial number of decryption key queries. For the j -th decryption key query on the vector $\mathbf{y}^{(j)} = (y_i^{(j)})_{i \in I}$, $\mathcal{B}$ then calculates the secret key $\text{sk}_{\mathbf{y}^{(j)}} \leftarrow \text{ulPFE.KeyExtract}(\text{Mpk}, \text{Msk}, \mathbf{y}^{(j)})$ and provides the secret key back to $\mathcal{A}$. $\mathcal{A}$ provides two different challenge vectors $\mathbf{x}^{(0)} = (x_i^{(0)})_{i \in I_0^*}$ and $\mathbf{x}^{(1)} = (x_i^{(1)})_{i \in I_1^*}$ with $|I_0^*| = |I_1^*| = \ell$ to $\mathcal{B}$ such that $\langle \mathbf{x}^{(0)}, \mathbf{y}^{(j)} \rangle = \langle \mathbf{x}^{(1)}, \mathbf{y}^{(j)} \rangle$ is satisfied whenever $I_0^* = I_1^* = I \forall \mathbf{y}^{(j)} = (y_i^{(j)})_{i \in I}$ for which decryption key query has already been made. Then $\mathcal{B}$ selects a bit $\beta \xleftarrow{\$} \{0, 1\}$ uniformly at random and transmit the challenge ciphertext $\text{Ct}_{\mathbf{x}^{(\beta)}} = (c_1, c_2) \leftarrow \text{ulPFE.Encrypt}(\text{Mpk}, \mathbf{x}^{(\beta)})$ to $\mathcal{A}$ where $c_1 = \mathbf{A}^\top \mathbf{s} + e_1 \in \mathbb{Z}_q^m, c_2 = \mathbf{U}^\top \mathbf{W}^\top \mathbf{s} + \lfloor \frac{q}{K} \rfloor \mathbf{x}^{(\beta)} + e_2 \in \mathbb{Z}_q^\ell$ and $\mathbf{s} \xleftarrow{\$} \mathbb{Z}_q^n, e_1 \xleftarrow{\$} \mathcal{D}_{\mathbb{Z}^m, \psi}, e_2 \xleftarrow{\$} \mathcal{D}_{\mathbb{Z}^\ell, \psi}, \mathbf{U} = [\mathbf{u}_1 \| \mathbf{u}_2 \|, \dots, \mathbf{u}_{|I_\beta^*|}] \in \mathbb{Z}_q^{\ell_{\max} \times \ell}$ with $\mathcal{H}(I_\beta^*[i]) = \mathbf{u}_i$ for all $I_\beta^*[i] \in I_\beta^*$. The two real numbers σ, ψ are selected in a way that $\sigma \geq \omega(\sqrt{\log n})$ and $\psi \geq 2C_1\alpha q(\sqrt{m} + \sqrt{n} + \sqrt{\ell_{\max}})$ where q is a large prime and $\alpha \in (0, 1) \cap \mathbb{R}$ satisfying $q \geq 2K(\sigma\psi m Y \ell_{\max}^2 + Y \ell_{\max} \psi)$.

Game 1: $\mathcal{A}$ tries to predict the index set I_0^* or I_1^* that $\mathcal{B}$ uses to generate the challenger ciphertext $\text{Ct}_{\mathbf{x}^{(\beta)}}$ linked to the vectors $\mathbf{x}^{(0)}$ and $\mathbf{x}^{(1)}$ respectively. An exponential security loss for $\mathcal{A}$ would result from guessing the directly index set I_0^* or I_1^* used to generate the challenge ciphertext $\text{Ct}_{\mathbf{x}^{(\beta)}}$. Rather, $\mathcal{A}$ selects the index of RO queries on I_0^* and I_1^* to obtain the matrices $\mathbf{U}_0 \in \mathbb{Z}_q^{\ell_{\max} \times |I_0^*|}$ and $\mathbf{U}_1 \in \mathbb{Z}_q^{\ell_{\max} \times |I_1^*|}$ respectively. For each index set query, it takes $\frac{1}{Q_{\text{ro}}^2}$ security loss where Q_{ro} is the total number of queries sent to the RO $\mathcal{H}$.

The following two scenarios are taken into consideration by $\mathcal{B}$ based on the challenge index sets I_0^* and I_1^* .

- **Case I** ($I_0^* = I_1^*$), $\mathcal{B}$ performs the ALS-IPFE reduction to demonstrate the security of our scheme approach.
- **Case II** ($I_0^* \neq I_1^*$), $\mathcal{B}$ employs the LWE reduction directly.

Case I ($I_0^* = I_1^*$)

$\mathcal{B}$ shows that the challenge ciphertext which is encrypting either message $\mathbf{x}^{(0)}$ or message $\mathbf{x}^{(1)}$ is indistinguishable by utilizing ALS-IPFE's security as described in subsection 2.3. When interacting with ALS-IPFE challenger $\mathcal{C}$, $\mathcal{B}$ is regarded as an intermediate adversary. We demonstrate that this game is reduced to the Adp-IND security of ALS-IPFE with the challenger's public key $\text{Mpk}_{\text{ALS}} = (\text{pp}_{\text{ALS}}, \mathbf{A}_{\text{ALS}}, \mathbf{D}_{\text{ALS}})$ with $\text{pp} = (n, q, m, \ell, X, Y, K, \psi, \alpha, \sigma)$, $\mathbf{A}_{\text{ALS}} \in \mathbb{Z}_q^{n \times m}$ and $\mathbf{D}_{\text{ALS}} \in \mathbb{Z}_q^{n \times \ell}$ along with $|I_0^*| = |I_1^*| = \ell$ fulfilling $\mathbf{D}_{\text{ALS}} = \mathbf{A}_{\text{ALS}} \mathbf{Z}_{\text{ALS}}$ and $\mathbf{Z}_{\text{ALS}} \in \mathcal{D}_{\mathbb{Z}^m \times \ell, \sigma}$ is kept secret to $\mathcal{C}$. $\mathcal{B}$ simulates $\mathcal{A}$ in the following manner using the index sets $I_0^* = I_1^* = I^*$ (say).

Setup

$\mathcal{B}$ sets $\mathbf{A} = \mathbf{A}_{\text{ALS}}$ rather than selecting the matrix $\mathbf{A}$ from $\mathbb{Z}_q^{n \times m}$ as in Game 0. Every other parameter is formed just like in Game 0.

– After querying the RO for I with $|I| = \ell$, a random matrix $\mathbf{U}_I = [\mathbf{u}_1 \| \mathbf{u}_2 \| \dots \| \mathbf{u}_{|I|}] \xleftarrow{\$} \{-1, 1\}^{\ell_{\max} \times \ell}$ is chosen by $\mathcal{B}$ and sets $\mathbf{u}_i = \mathcal{H}(I[i]) \in \{-1, 1\}^{\ell_{\max}}$ to program the RO $\mathcal{H}$. The uIPFE #1 challenger $\mathcal{B}$ returns $\text{sk}_{\mathbf{y}^{(j)}} = (\mathbf{y}^{(j)}, (\mathbf{y}^{(j)})^\top \mathbf{U}_I^\top \mathbf{Z}_I^\top)$ for the decryption key queries on the vector $\mathbf{y}^{(j)} = (y_i^{(j)})_{i \in I}$ with $|I| = \ell$. This distribution is identical to that of Game 0.

Challenge ciphertext

The challenge message vector pair $(\mathbf{x}^{(0)}, \mathbf{x}^{(1)})$ is sent by $\mathcal{A}$ to uIPFE #1 challenger $\mathcal{B}$ who then transmits it to ALS-IPFE challenger $\mathcal{C}$. Then $\mathcal{B}$ gives the challenge ciphertext $\text{Ct}^{(\beta)} = (\mathbf{c}_1, \mathbf{c}_2)$ to $\mathcal{A}$ after receiving it from $\mathcal{C}$. Here $\mathbf{c}_1 = \mathbf{A}_{\text{ALS}}^\top \mathbf{s} + \mathbf{e}_1 \in \mathbb{Z}_q^m$, $\mathbf{c}_2 = \mathbf{D}_{\text{ALS}}^\top \mathbf{s} + \lfloor \frac{q}{K} \rfloor \mathbf{x}^{(\beta)} + \mathbf{e}_2$ where $\mathbf{e}_1 \xleftarrow{\$} \mathcal{D}_{\mathbb{Z}^m, \psi}$, $\mathbf{e}_2 \xleftarrow{\$} \mathcal{D}_{\mathbb{Z}^\ell, \psi}$.

Decryption keys

$\mathcal{B}$ sends the vector $\mathbf{y}^{(j)}$ to $\mathcal{C}$ for the j -th decryption key query on $\mathbf{y}^{(j)} = (y_i^{(j)})_{i \in I}$ with $I = I^*$ (or $\neq I^*$). Then $\mathcal{C}$ generates the decryption key $\text{sk}_{\mathbf{y}^{(j)}} = (\mathbf{y}^{(j)}, (\mathbf{y}^{(j)})^\top \mathbf{Z}_{\text{ALS}}^\top)$ (here $\mathbf{Z}_{\text{ALS}} \in \mathbb{Z}^{m \times \ell}$ is the master secret key of $\mathcal{C}$ satisfying $\mathbf{D}_{\text{ALS}} = \mathbf{A}_{\text{ALS}} \mathbf{Z}_{\text{ALS}}$) and transmits it to $\mathcal{B}$ who then forwards it to $\mathcal{A}$. Consider the decryption key as $\text{sk}_{\mathbf{y}^{(j)}} = (\mathbf{y}^{(j)}, (\mathbf{y}^{(j)})^\top \mathbf{U}_{I^*}^\top \mathbf{Z}^\top)$ where $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, $\mathbf{U}_{I^*} = [\mathbf{u}_1 \| \mathbf{u}_2 \| \dots \| \mathbf{u}_{|I^*|}] \in \mathbb{Z}_q^{\ell_{\max} \times \ell}$ with $\mathcal{H}(I^*[i]) = \mathbf{u}_i \in \{-1, 1\}^{\ell_{\max}}$ for all $I^*[i] \in I^*$ and $\mathbf{Z} \in \mathcal{D}_{\mathbb{Z}^{m \times \ell_{\max}}, \sigma}$ which has a distribution identical to that of Game 0. The decryption key $\text{sk}_{\mathbf{y}^{(j)}} = (\mathbf{y}^{(j)}, (\mathbf{y}^{(j)})^\top \mathbf{Z}_{\text{ALS}}^\top)$, on the other hand, has the same distribution as in Game 1 with $\mathbf{D}_{\text{ALS}} = \mathbf{A}_{\text{ALS}} \mathbf{Z}_{\text{ALS}}$ where $\mathbf{D}_{\text{ALS}} \in \mathbb{Z}_q^{n \times \ell}$, $\mathbf{A}_{\text{ALS}} \in \mathbb{Z}_q^{n \times m}$ and $\mathbf{Z}_{\text{ALS}} \in \mathcal{D}_{\mathbb{Z}^{m \times \ell}, \sigma}$. In Game 1, $\mathcal{B}$ sets $\mathbf{A} = \mathbf{A}_{\text{ALS}}$ and $\mathbf{WU}_{I^*} = \mathbf{D}_{\text{ALS}} = \mathbf{A}_{\text{ALS}} \mathbf{Z}_{\text{ALS}}$ for $I = I^*$ in Game 1 by implicitly setting $\mathbf{ZU}_{I^*} = \mathbf{Z}_{\text{ALS}}$. In both Games 0 and 1, the decryption keys are distributed in the same way. We can see that $\text{Adv}_{\mathcal{A}, \text{Game}_1}(\lambda) \leq \text{Adv}_{\mathcal{B}, \text{ALS-IPFE}}^{\text{Adp-IND}}(\lambda)$ and from the following argument $|\text{Adv}_{\mathcal{A}, \text{Game}_1}(\lambda) - \text{Adv}_{\mathcal{A}, \text{Game}_0}(\lambda)| \leq \text{Adv}_{\mathcal{B}, \text{ALS-IPFE}}^{\text{Adp-IND}}(\lambda)$.

• For $I = I^*$ (or $\neq I^*$), we must show that the distribution of ALS-IPFE's public keys is statistically similar to that of honestly extracted keys, as in Game 0. While in Game 0, $\mathbf{W}$ takes the form $\mathbf{W} = \mathbf{AZ}$ where $\mathbf{Z} \xleftarrow{\$} \mathcal{D}_{\mathbb{Z}^{m \times \ell_{\max}}, \sigma}$ and $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, the matrix the matrix $\mathbf{WU}_{I^*} = \mathbf{D}_{\text{ALS}}$ in ALS-IPFE takes the form $\mathbf{D}_{\text{ALS}} = \mathbf{A}_{\text{ALS}} \mathbf{Z}_{\text{ALS}}$ where $\mathbf{Z}_{\text{ALS}} \xleftarrow{\$} \mathcal{D}_{\mathbb{Z}^{m \times \ell}, \sigma}$ and $\mathbf{A}_{\text{ALS}} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$. Given $\mathbf{A} = \mathbf{A}_{\text{ALS}}$ and $\mathbf{WU}_{I^*} = \mathbf{D}_{\text{ALS}}$, it is evident that the public keys of ALS-IPFE and Game 0 are connected. The RO $\mathcal{H}$ is programmed by $\mathcal{B}$ such that $\mathbf{WU}_{I^*} = \mathbf{D}_{\text{ALS}}$. The distribution of $\mathbf{ZU}_{I^*}$ is equal to that of $\mathbf{Z}_{\text{ALS}} \xleftarrow{\$} \mathcal{D}_{\mathbb{Z}^{m \times \ell}, \sigma}$ according to Lemma 2 where σ of ALS-IPFE is set as σ of ALS-IPFE = σ of uIPFE $\cdot \sqrt{\ell_{\max}}$. Consequently, the matrix $\mathbf{WU}_{I^*}$ is not statistically close to uniform. Nevertheless, this is not a problem in our case because the distribution of $\mathbf{WU}_{I^*}$ is uniform distributed given that it is in the form $\mathbf{AZU}_{I^*} = \mathbf{A}_{\text{ALS}} \mathbf{Z}_{\text{ALS}}$ with $\mathbf{A}_{\text{ALS}} \in \mathbb{Z}_q^{n \times m}$ and $\mathbf{Z}_{\text{ALS}} \xleftarrow{\$} \mathcal{D}_{\mathbb{Z}^{m \times \ell}, \sigma}$ ($\mathbf{ZU}_{I^*}$ is implicitly set as $\mathbf{Z}_{\text{ALS}}$). Therefore, we can conclude that $\text{Dist}_0 = (\mathbf{A}_{\text{ALS}}, \mathbf{Z}_{\text{ALS}}, \mathbf{D}_{\text{ALS}})$ is statistically close to $\text{Dist}_1 = (\mathbf{A}, \mathbf{ZU}_{I^*}, \mathbf{WU}_{I^*})$ with uniform $\mathbf{U}_{I^*}$. Similar to the decryption keys in Game 0 corresponding to the vector $\mathbf{y}^{(j)} = (y_i^{(j)})_{i \in I}$, the keys in Game 1 (produced by $\mathcal{C}$) are distributed statistically close to honestly extracted keys.

Case II ($I_0^* \neq I_1^*$): $\mathcal{A}$ cannot get the decryption key for either the vector $(y_i^{(j)})_{i \in I_0^*}$ or $(y_i^{(j)})_{i \in I_1^*}$ in Adp-IND security model since these keys can readily discriminate between the encryption of $\mathbf{x}^{(0)} = (x_i^{(0)})_{i \in I_0^*}$ and $\mathbf{x}^{(1)} = (x_i^{(1)})_{i \in I_1^*}$. As a result, $\mathcal{A}$ is unable to query for

decryption keys associated with the index sets I_0^* and I_1^* . We show below that the challenger can immediately reduce the problem to LWE for the matrix

$$\mathbf{E} = \begin{bmatrix} \mathbf{A} & \mathbf{W}\mathbf{U}_0 & \mathbf{W}\mathbf{U}_1 \end{bmatrix} \in \mathbb{Z}_q^{n \times (m + \ell_0 + \ell_1)}$$

where $|I_0^*| = \ell_0$ and $|I_1^*| = \ell_1$, $\mathbf{A} \xleftarrow{\$} \mathbb{Z}_q^{n \times m}$, $\mathbf{W} = \mathbf{A}\mathbf{Z} \in \mathbb{Z}_q^{n \times \ell_{\max}}$, $\mathbf{U}_0 = [\mathbf{u}_1 \| \mathbf{u}_2 \| \dots \| \mathbf{u}_{\ell_0}] \in \{-1, 1\}^{\ell_{\max} \times \ell_0}$ with $\mathcal{H}(I_0^*[i]) = \mathbf{u}_i \in \{-1, 1\}^{\ell_{\max}}$ for all $I_0^*[i] \in I_0^*$ and $\mathbf{U}_1 = [\mathbf{u}'_1 \| \mathbf{u}'_2 \| \dots \| \mathbf{u}'_{\ell_1}] \in \{-1, 1\}^{\ell_{\max} \times \ell_1}$ with $\mathcal{H}(I_1^*[i]) = \mathbf{u}'_i \in \{-1, 1\}^{\ell_{\max}}$ for all $I_1^*[i] \in I_1^*$. The LWE challenger selects column vector $\mathbf{d} \xleftarrow{\$} \mathbb{Z}_q^n$, $\mathbf{g} = (g_1, g_2^{(0)}, g_2^{(1)}) \xleftarrow{\$} \mathcal{D}_{\mathbb{Z}^{m + \ell_0 + \ell_1}, \psi}$ to produce the tuple $\mathbf{z} = (c_1, c_2^{(0)}, c_2^{(1)})$ from the matrix $\mathbf{E}$. $\mathbf{E}$ is a random matrix as $\mathcal{H}$ is modelled as the RO.

Note that, $\mathbf{E}$ is a random matrix since $\mathcal{H}$ is modelled here as the RO.

– For any $I \notin \{I_0^*, I_1^*\}$ query to the RO $\mathcal{H}$, the challenger selects a matrix $\mathbf{U}_I \xleftarrow{\$} \{-1, 1\}^{\ell_{\max} \times \ell}$ with $|I| = \ell$ and in order to program the RO $\mathcal{H}$ so that the output $\mathbf{U}_I$ is calculated as $\mathbf{U}_I = [\mathbf{u}_1 \| \mathbf{u}_2 \| \dots \| \mathbf{u}_{|I|}]$ with $\mathbf{u}_i = \mathcal{H}(I[i])$, $\forall I[i] \in I$. The adversary obtains the decryption key $\mathbf{sk}_{\mathbf{y}^{(j)}} = (\mathbf{y}^{(j)}, (\mathbf{y}^{(j)})^\top \mathbf{U}_I^\top \mathbf{Z}^\top)$ corresponding to the vector $\mathbf{y}^{(j)} = (y_i^{(j)})_{i \in I}$. To response the challenge query for the vector $\mathbf{x}^{(\beta)} = (x_i^{(\beta)})_{i \in I_\beta^*}$, the challenger now selects a random bit $\beta \xleftarrow{\$} \{0, 1\}$. $\mathcal{C}$ gives back $\mathbf{ct}_{\mathbf{x}^{(0)}} = (c_1, c_2^{(0)} + \lfloor \frac{q}{K} \rfloor \mathbf{x}^{(0)})$ for $\beta = 0$ and $\mathbf{ct}_{\mathbf{x}^{(1)}} = (c_1, c_2^{(1)} + \lfloor \frac{q}{K} \rfloor \mathbf{x}^{(1)})$ for $\beta = 1$.

The distribution of the challenge ciphertext is $\mathbf{Ct}_{\mathbf{x}^{(\beta)}} = (c_1 = \mathbf{A}^\top \mathbf{d} + g_1, c_2 = \tilde{c} + \lfloor \frac{q}{K} \rfloor \mathbf{x}^{(\beta)})$ when the LWE oracle produces a random pair $(\mathbf{W}\mathbf{U}_\beta, \tilde{c})$ where $\mathbf{W} \in \mathbb{Z}_q^{n \times \ell_{\max}}$, $\mathbf{U}_\beta \xleftarrow{\$} \{-1, 1\}^{\ell_{\max} \times \ell}$ and $\tilde{c} \xleftarrow{\$} \mathbb{Z}_q^\ell$. This is same as in Game 1. When the pair $(\mathbf{W}\mathbf{U}_\beta, c_2^{(\beta)})$ is produced by the LWE oracle, with $\mathbf{W} \in \mathbb{Z}_q^{n \times \ell_{\max}}$, $\mathbf{U}_\beta \xleftarrow{\$} \{-1, 1\}^{\ell_{\max} \times \ell}$ and $c_2^{(\beta)} = \mathbf{W}\mathbf{U}_\beta \mathbf{d} + \mathbf{g}^{(\beta)}$ by calculating

$$\begin{aligned} \mathbf{c} &= \mathbf{E}^\top \mathbf{d} + \mathbf{g} = \begin{bmatrix} \mathbf{A} & \mathbf{W}\mathbf{U}_0 & \mathbf{W}\mathbf{U}_1 \end{bmatrix}^\top \mathbf{d} + (g_1, g_2^{(0)}, g_2^{(1)}) \\ &= (\mathbf{A}^\top \mathbf{d} + g_1, \mathbf{U}_0^\top \mathbf{W}^\top \mathbf{d} + g_2^{(0)}, \mathbf{U}_1^\top \mathbf{W}^\top \mathbf{d} + g_2^{(1)}) \\ &= (c_1, c_2^{(0)}, c_2^{(1)}) \in \mathbb{Z}_q^m \times \mathbb{Z}_q^{\ell_0} \times \mathbb{Z}_q^{\ell_1}. \end{aligned}$$

The challenge ciphertext's distribution is $\mathbf{Ct}_{\mathbf{x}^{(\beta)}} = (c_1 = \mathbf{A}^\top \mathbf{d} + g_1, c_2 = c_2^{(\beta)} + \lfloor \frac{q}{K} \rfloor \mathbf{x}^{(\beta)}) \in \mathbb{Z}_q^m \times \mathbb{Z}_q^{\ell_\beta}$ which is same as in Game 0. Consequently, we may write as follows:

$$|\text{Adv}_{\mathcal{A}, \text{Game}_1}(\lambda) - \text{Adv}_{\mathcal{A}, \text{Game}_0}(\lambda)| \leq \text{Adv}_{\mathcal{B}}^{\text{LWE}_{q, \alpha, n}}(\lambda).$$

Consequently, it can be said that

$$\text{Adv}_{\mathcal{A}}^{\text{Adp-IND}}(\lambda) \leq \frac{1}{Q_{\text{ro}}^2} \cdot \text{Adv}_{\mathcal{B}}^{\text{LWE}_{q, \alpha, n}}(\lambda)$$

where Q_{ro} is the total number of queries that RO has been asked. $\square$

5 Implementation Result

We have implemented the uIPFE [1] and our uIPFE #1 in programming language C and run it on IBM Power9, model 9006-22P. Our trapdoor sampling implementation

is based on [13] and pre-image sampling implementation is based on [16] and are independent of any external sources like MPFR or GMP. Multi-threading is used to make the implementation fast. For instance, we run the procedure `SamplePre` parallelly using loop parallelization inside the procedure `SamplePreimage` that reduces the runtime. `SamplePreimage` takes almost the same time as a single execution of the procedure `SamplePre` while running on a machine with multi-core via multi-threading using OpenMP. More specifically, we have used OpenMP API for multi-threading on IBM POWER9 with 32 cores rated at 3.8GHz. The same concept of loop parallelization is used for the procedure `GSampling`. It is evident that the overall runtime of `uIPFE` is primarily influenced by the `TrapdoorGen` and `SamplePreimage` algorithms. In contrast, our proposed scheme, `uIPFE #1`, eliminates the need for both trapdoor generation and preimage sampling, resulting in a considerably faster execution and superior overall performance compared to `uIPFE`. However, the encryption phase in `uIPFE #1` is marginally slower due to the presence of additional multiplication operations. The implementation code is available in <https://github.com/uIPFE-implementation>.

Table 3: Representative parameter sets for `uIPFE` and `uIPFE #1`

Parameter set	I	II	III	IV
n	5	5	10	20
m (<code>uIPFE</code>)	530	480	980	1760
m (<code>uIPFE #1</code>)	640	570	1160	2040
$\ell_{\max}$	5000	1000	800	100
X	8	8	8	8
Y	8	8	8	8
q (<code>uIPFE</code>)	4503599627370517($\approx 2^{53}$)	140737488355333($\approx 2^{48}$)	281474976710677($\approx 2^{49}$)	8796093022237($\approx 2^{44}$)
q (<code>uIPFE #1</code>)	$\approx 2^{64}$	144115188075855881($\approx 2^{57}$)	288230376151711813($\approx 2^{58}$)	1125899906842679($\approx 2^{51}$)
ψ	2.236	2.236	3.162	4.472
α (<code>uIPFE</code>)	2.587×10^{-15}	1.424×10^{-13}	8.952×10^{-14}	4.505×10^{-12}
α (<code>uIPFE #1</code>)	6.169×10^{-19}	1.344×10^{-16}	8.374×10^{-17}	3.330×10^{-14}
σ	83.29	83.29	83.29	83.29

Figures 5, 6, 7 and 8 illustrate the performance comparison between the `uIPFE` scheme proposed by Dutta et al. [1] and our proposed `uIPFE #1`, corresponding to the parameter sets I, II, III and IV, respectively, as listed in Table 3. From these figures 5 - 8, it is evident that the overall runtime of the scheme in [1] is higher than that of our `uIPFE #1`. In particular, our key extraction and decryption algorithms execute significantly faster, whereas the encryption algorithm of [1] performs slightly better than that of our implementation.

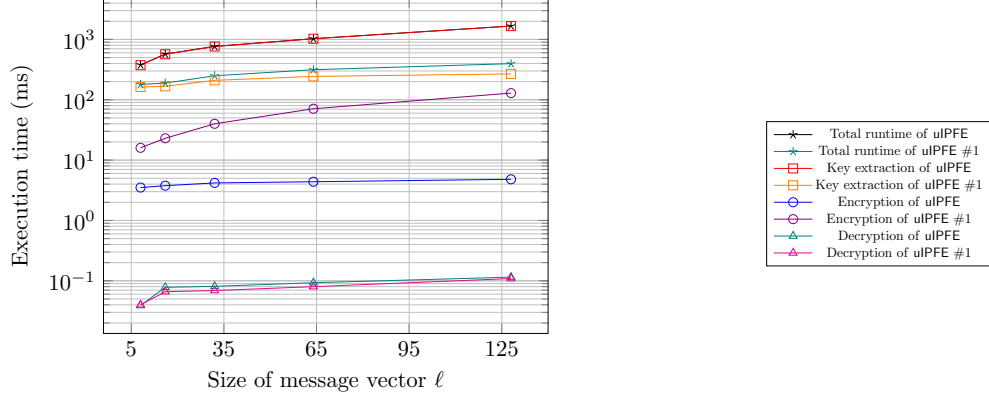


Fig. 5: Execution time comparison of `uIPFE.KeyExtract`, `uIPFE.Encrypt`, `uIPFE.Decrypt` and total runtime of `uIPFE` [1] and `uIPFE #1` for $\ell = 8, 16, 32, 64, 128$ with the choice of parameter set **I** given in Table 3

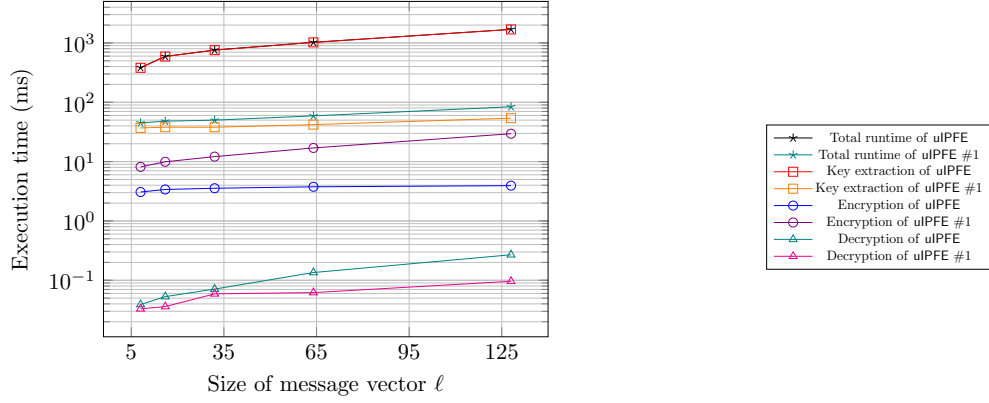


Fig. 6: Execution time comparison of `uIPFE.KeyExtract`, `uIPFE.Encrypt`, `uIPFE.Decrypt` and total runtime of `uIPFE` [1] and `uIPFE #1` for $\ell = 8, 16, 32, 64, 128$ with the choice of parameter set **II** given in Table 3

6 Conclusion

This work presents the implementation of the scheme `uIPFE` of Dutta et al. [1] and construction of a new scheme strict `uIPFE #1` without employing any pairing operations. Our scheme `uIPFE #1` achieves post-quantum security against adaptive adversaries using a random oracle without using trapdoor generation and pre-image sampling algorithms. From a practical perspective, this primitive holds greater significance in scenarios where the vectors' length is not known beforehand. Thus, our implementation of the `uIPFE` of Dutta et al. [1] and our construction of `uIPFE #1` are capable of encrypting or decrypting vectors of any size using the same public parameters. In

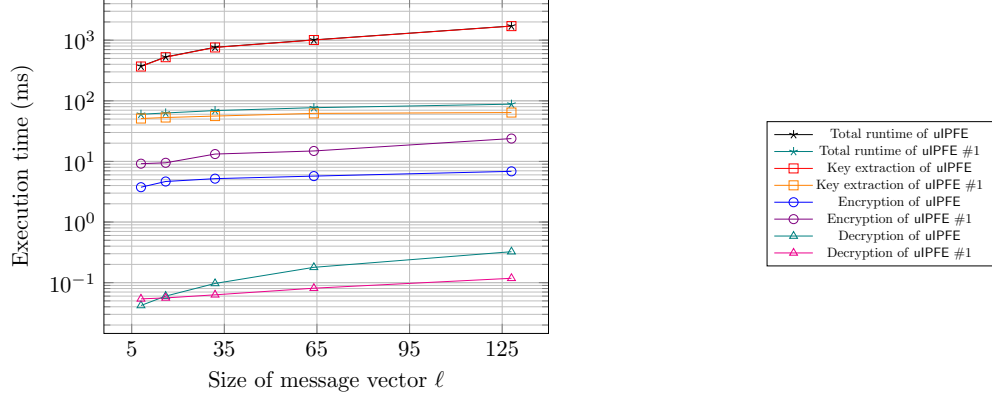


Fig. 7: Execution time comparison of uIPFE.KeyExtract, uIPFE.Encrypt, uIPFE.Decrypt and total runtime of uIPFE [1] and uIPFE #1 for $\ell = 8, 16, 32, 64, 128$ with the choice of parameter set **III** given in Table 3

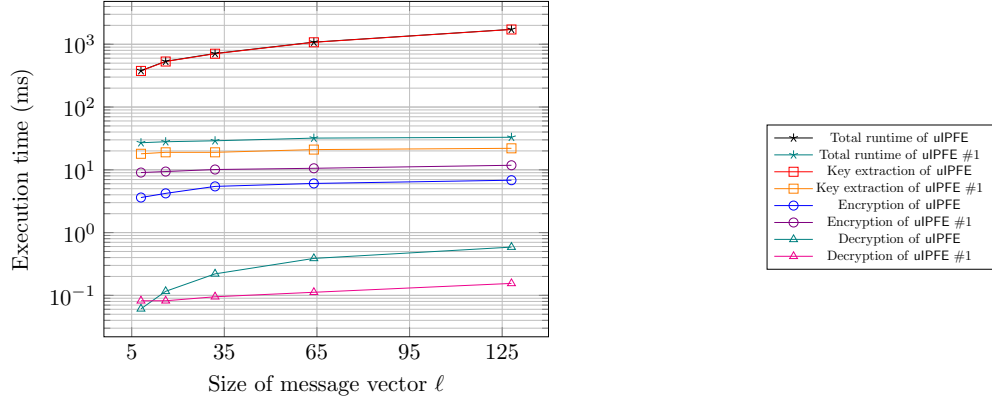


Fig. 8: Execution time comparison of uIPFE.KeyExtract, uIPFE.Encrypt, uIPFE.Decrypt and total runtime of uIPFE [1] and uIPFE #1 for $\ell = 8, 16, 32, 64, 128$ with the choice of parameter set **IV** given in Table 3

addition, we evaluate both the schemes uIPFE and uIPFE #1 based on their execution time and give the implementation outcome. However, constructing such a uIPFE for multi-user systems where multiple users may collide, remains an open problem. Additionally, designing the uIPFE within the standard model based on post-quantum assumptions may pose a significant challenge.

Acknowledgements. The work was partially supported by the Space Applications Centre (SAC), Indian Space Research Organisation (ISRO), (Project ID: STC0322) of the Government of India. We express our sincere gratitude to Subhranil Dutta and Jayashree Dey for their involvement on this work.

Declarations

- Funding : The work received partial financial support from the Space Applications Centre (SAC), Indian Space Research Organisation (ISRO), Government of India, under Project ID: STC0322.
- Conflict of interest/Competing interests : The authors confirm that there are no financial or personal relationships that could have inappropriately influenced the results or interpretations presented in this paper.
- Ethics approval and consent to participate : The authors further confirm that the results, data and figures presented in this manuscript are original, have not been published previously and are not under review by any other journal or publisher.
- Consent for publication : All authors have reviewed the manuscript and consented to its submission in the present form.
- Data availability : No data was utilized for the study outlined in the article.
- Materials availability : Not applicable
- Code availability : The implementation code is available in <https://github.com/uIPFE-implementation>.
- Author contribution : **Suprava Roy**: Writing - original draft, Writing review & editing, Conceptualization, Formal analysis, Methodology. **Ratna Dutta**: Writing review & editing, Supervision, Formal analysis, Methodology, Conceptualization.

References

- [1] Dutta, S., Dutta, R., Mukhopadhyay, S.: Constructing pairing free unbounded inner product functional encryption schemes with unbounded inner product policy. In: International Conference on Information Technology and Communications Security, pp. 102–116 (2022). Springer
- [2] Agrawal, S., Libert, B., Stehlé, D.: Fully secure functional encryption for inner products, from standard assumptions. In: Annual International Cryptology Conference, pp. 333–362 (2016). Springer
- [3] Boneh, D., Sahai, A., Waters, B.: Functional encryption: Definitions and challenges. In: Theory of Cryptography Conference, pp. 253–273 (2011). Springer
- [4] Abdalla, M., Bourse, F., De Caro, A., Pointcheval, D.: Simple functional encryption schemes for inner products. In: IACR International Workshop on Public Key Cryptography, pp. 733–751 (2015). Springer
- [5] Benhamouda, F., Bourse, F., Lipmaa, H.: Cca-secure inner-product functional encryption from projective hash functions. In: Fehr, S. (ed.) Public-Key Cryptography – PKC 2017, pp. 36–66. Springer, Berlin, Heidelberg (2017)
- [6] Abdalla, M., Catalano, D., Gay, R., Ursu, B.: Inner-product functional encryption with fine-grained access control. In: International Conference on the Theory and Application of Cryptology and Information Security, pp. 467–497 (2020). Springer

- [7] Abdalla, M., Gay, R., Raykova, M., Wee, H.: Multi-input inner-product functional encryption from pairings. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques, pp. 601–626 (2017). Springer
- [8] Datta, P., Okamoto, T., Tomida, J.: Full-hiding (unbounded) multi-input inner product functional encryption from the k -linear assumption. In: Abdalla, M., Dahab, R. (eds.) Public-Key Cryptography – PKC 2018, pp. 245–277. Springer, Cham (2018)
- [9] Tomida, J., Takashima, K.: Unbounded inner product functional encryption from bilinear maps. In: Advances in Cryptology–ASIACRYPT 2018: 24th International Conference on the Theory and Application of Cryptology and Information Security, Brisbane, QLD, Australia, December 2–6, 2018, Proceedings, Part II 24, pp. 609–639 (2018). Springer
- [10] Dufour-Sans, E., Pointcheval, D.: Unbounded inner-product functional encryption with succinct keys. In: International Conference on Applied Cryptography and Network Security, pp. 426–441 (2019). Springer
- [11] Dutta, S., Dutta, R., Mukhopadhyay, S.: Securing data in the cloud using pairing-free inner product functional encryption with unbounded vector size. *Theoretical Computer Science* **1031**, 115085 (2025)
- [12] Dowerah, U., Dutta, S., Hartmann, F., Mitrokotsa, A., Mukherjee, S., Pal, T.: Sacfe: Secure access control in functional encryption with unbounded data. In: 2024 IEEE 9th European Symposium on Security and Privacy (EuroS&P), pp. 860–882 (2024). IEEE
- [13] Gentry, C., Peikert, C., Vaikuntanathan, V.: Trapdoors for hard lattices and new cryptographic constructions. In: Proceedings of the Fortieth Annual ACM Symposium on Theory of Computing, pp. 197–206 (2008)
- [14] Micciancio, D., Peikert, C.: Trapdoors for lattices: Simpler, tighter, faster, smaller. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques, pp. 700–718 (2012). Springer
- [15] Bert, P., Eberhart, G., Prabel, L., Roux-Langlois, A., Sabt, M.: Implementation of lattice trapdoors on modules and applications. In: Post-Quantum Cryptography: 12th International Workshop, PQCrypto 2021, Daejeon, South Korea, July 20–22, 2021, Proceedings 12, pp. 195–214 (2021). Springer
- [16] Gür, K.D., Polyakov, Y., Rohloff, K., Ryan, G.W., Savas, E.: Implementation and evaluation of improved gaussian sampling for lattice trapdoors. In: Proceedings of the 6th Workshop on Encrypted Computing & Applied Homomorphic Cryptography, pp. 61–71 (2018)

- [17] Ducas, L., Nguyen, P.Q.: Faster gaussian lattice sampling using lazy floating-point arithmetic. In: International Conference on the Theory and Application of Cryptology and Information Security, pp. 415–432 (2012). Springer
- [18] Genise, N., Micciancio, D.: Faster gaussian sampling for trapdoor lattices with arbitrary modulus. In: Advances in Cryptology–EUROCRYPT 2018: 37th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Tel Aviv, Israel, April 29–May 3, 2018 Proceedings, Part I 37, pp. 174–203 (2018). Springer
- [19] Agrawal, S., Boneh, D., Boyen, X.: Efficient lattice (h) ibe in the standard model. In: Annual International Conference on the Theory and Applications of Cryptographic Techniques, pp. 553–572 (2010). Springer
- [20] Agrawal, S., Boneh, D., Boyen, X.: Lattice basis delegation in fixed dimension and shorter-ciphertext hierarchical ibe. In: Advances in Cryptology–CRYPTO 2010: 30th Annual Cryptology Conference, Santa Barbara, CA, USA, August 15–19, 2010. Proceedings 30, pp. 98–115 (2010). Springer
- [21] Micciancio, D., Regev, O.: Worst-case to average-case reductions based on gaussian measures. *SIAM journal on computing* **37**(1), 267–302 (2007)
- [22] Micciancio, D., Regev, O.: Worst-case to average-case reductions based on gaussian measures. *SIAM Journal on Computing* **37**(1), 267–302 (2007)
- [23] Luo, F., Wang, F., Wang, K., Chen, K.: A more efficient leveled strongly-unforgeable fully homomorphic signature scheme. *Information Sciences* **480**, 70–89 (2019)
- [24] Regev, O.: On lattices, learning with errors, random linear codes, and cryptography, 2005. In: STOC, pp. 84–93 (2005). ACM
- [25] Peikert, C., *et al.*: A decade of lattice cryptography. *Foundations and trends® in theoretical computer science* **10**(4), 283–424 (2016)
- [26] Zheng, Z., Tian, K., Liu, F.: Fully Homomorphic Encryption, pp. 143–174. Springer, Singapore (2023). https://doi.org/10.1007/978-981-19-7644-5_6